AKTU

B.Tech I-Year

GATEWAY CLASSES

PPS: Programming For Problem Solving

ONE SHOT Revision

(Crash Course)

Unit-2

Arithmetic Expressions and Precedence

Video Lectures I Pdf Notes I PYQs

By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

PROGRAMMING FOR PROBLEM SOLVING

UNIT-2

AKTU SYLLABUS

Arithmetic Expressions and Precedence : Operators and Expression Using Numeric and Relational Operators, Mixed Operands, Type Conversion, Logical Operators, Bit Operations, Assignment Operator, Operator precedence and Associatively.

Conditional Branching: Applying if and Switch Statements, Nesting if and Else and Switch.

AKTU

B.Tech I-Year

GW GATEWAY CLASSES

PPS: Programming For Problem Solving

UNIT-2

Lecture-1

Topics to be Covered

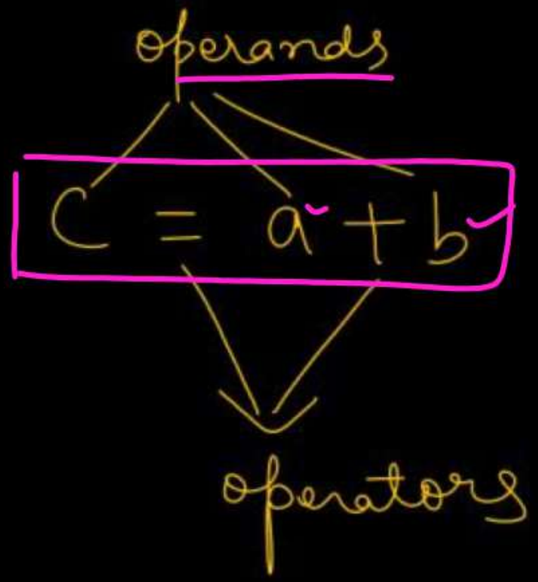
- Operator in C
  - Arithmetic operator
    - Increment/Decrement operator
  - Relational operator
  - Logical operator
- AKTU PYQs

By Dr. Nidhi Parashar Ma'am
- M.Tech Gold Medalist
- Net Qualified

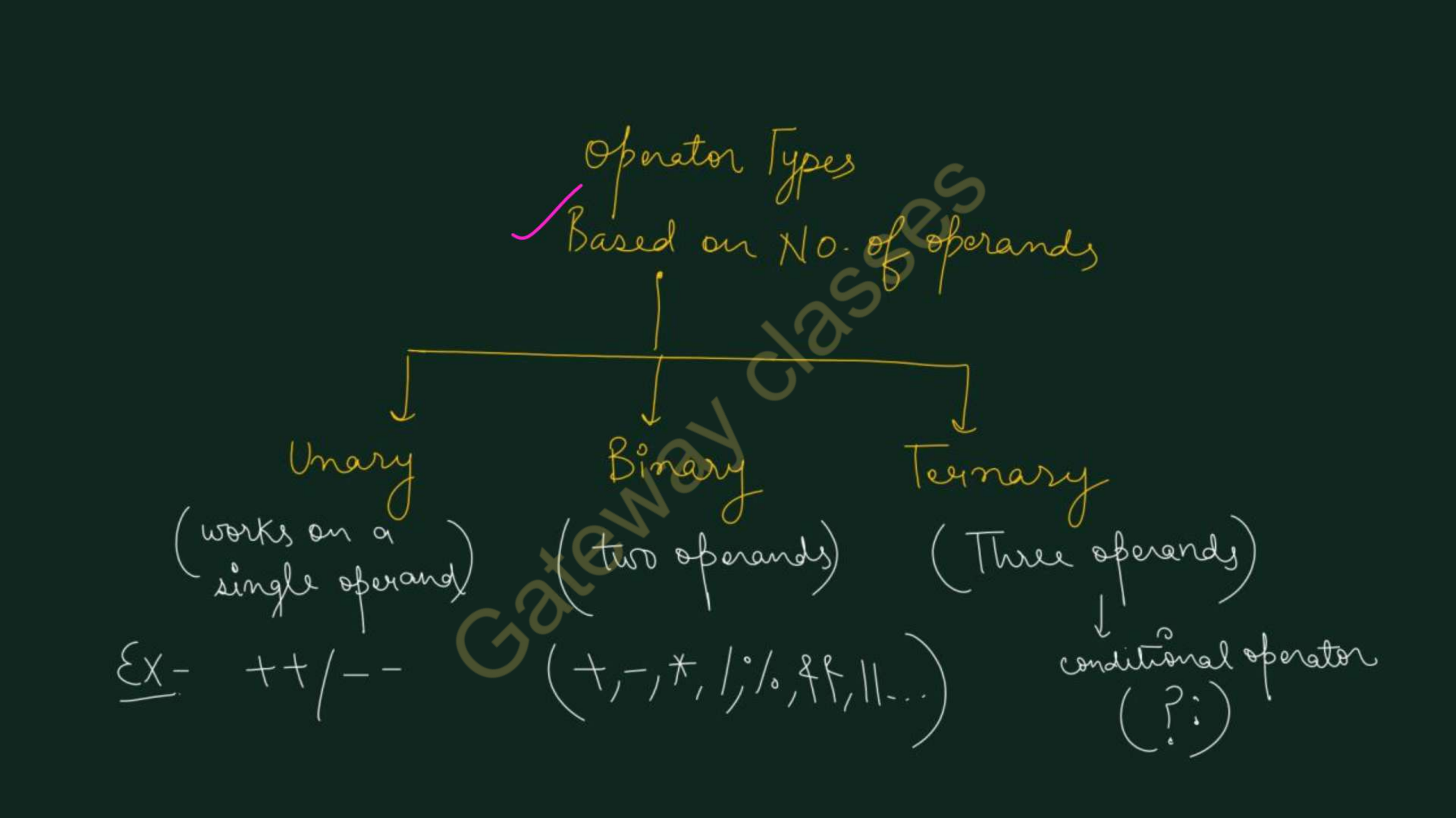
Video Lectures I Pdf Notes I PYQs

Operators in C

- Operators are symbols that represent operations to be performed on one or more operands.
- Tells the compiler to perform specific mathematical or logical manipulations
- The values and variables used with operators are called operands.



	Operators	Type
Unary Operator →	++, --	Unary Operator
Binary Operator	+, -, *, /, %	Arithmetic Operator
	<, <=, >, >=, ==, !=	Rational Operator
	&&, ||, !	Logical Operator
	&, |, <<, >>, ~, ^	Bitwise Operator
	=, +=, -=, *=, /=, %=	Assignment Operator
Ternary Operator →	?:	Ternary or Conditional Operator



Types of operators

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Bitwise Operators
- Assignment Operators
- Conditional operators
- Assignment Operator
- Comma operator
- sizeof() operator

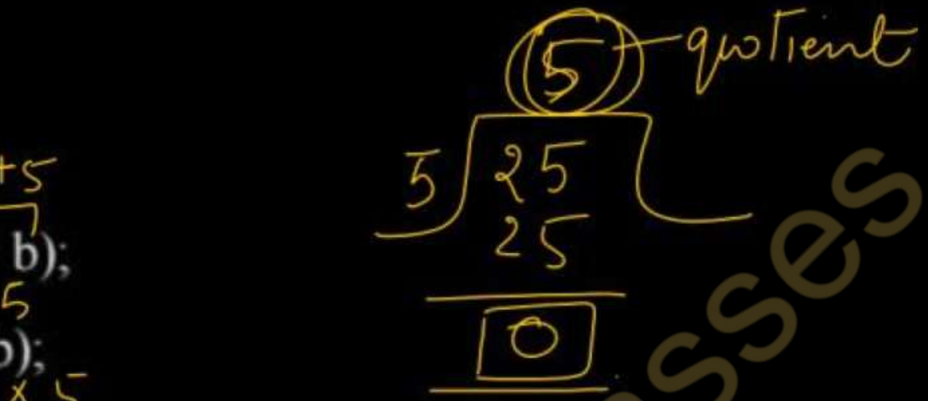
Arithmetic Operators in C

Perform arithmetic/mathematical operations on operands.

S. No.	Symbol	Operator	Description	Syntax
1	+	Plus	Adds two numeric values.	a + b
2	-	Minus	Subtracts right operand from left operand.	a – b
3	*	Multiply	Multiply two numeric values.	a * b
4	/	Divide	Divide two numeric values.	a / b
5	%	Modulus	Returns the remainder after diving the left operand with the right operand.	a % b
6	+	Unary Plus	Used to specify the positive values.	+a
7	-	Unary Minus	Flips the sign of the value.	-a
8	++	Increment	Increases the value of the operand by 1.	a++
9	--	Decrement	Decreases the value of the operand by 1.	a--

```c
#include <stdio.h>
int main()
{
    int a = 25, b = 5;
    printf("a + b = %d\n", a + b);
    printf("a - b = %d\n", a - b);
    printf("a * b = %d\n", a * b);
    printf("a / b = %d\n", a / b);
    printf("a % b = %d\n", a % b);
    printf("+a = %d\n", +a);
    printf("-a = %d\n", -a);
    printf("a++ = %d\n", a++);
    printf("a-- = %d\n", a--);
    return 0;
}
```

25+5
25-5
25 x 5
25/5 → it gives quotient
a % b → it gives remainder



printf("%d", 9); //25

OUTPUT:

```
a + b = 30
a - b = 20
a * b = 125
a / b = 5
a % b = 0
+a = 25
-a = -25
a++ = 25
a-- = 26
```

Modulo - Division Operator

① Gives the remainder

```
int main()
{
    int x = 5, y = 2, c;
    c = x % y;          // 5 % 2 => 1
    printf("%d", c);    // It will print 1
    return 0;
}
```

②

```
int main()
{
    int a, b, c, d;
    a = -5 % 2;   // -1
    b = -5 % -2;  // -1
    c = 5 % -2;   // 1
    d = 5 % 2;    // 1
    printf("%d %d %d %d", a, b, c, d); // -1 -1 1 1
    return 0;
}
```

```
int main()
{
    float x = 2.5, y = 0.5, z;
    z = x % y;  // compiler will produce an error
    printf("%f", z);
}
```

③ % does not work on float/double values.

→ It works only on int and char types only

Increment/Decrement Operators(++/ --) (Unary)

① Increment (++)

a++ / ++a

Means

a = a+1

② Decrement operator

a-- / --a

Means

a = a-1

```
int a = 1;
a++;
printf("%d", a);   //2
a--;
printf("%d", a);   //1
```

++a / a++

a = 2

a++

a = a+1

a = 2+1

⇒ a = 3

++a

(pre increment)

a++

(post increment)

--a

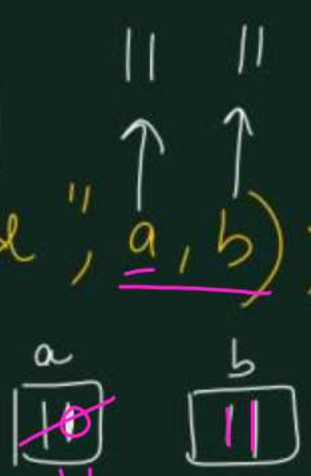
(pre)

a--

(post)

Diff b/w Pre-Increment V. Imp and Post-Increment V. Imp

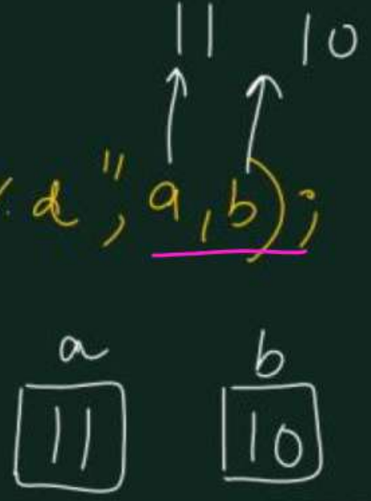
```
#include<stdio.h>
int main()
{ int a=10,b;
  b=++a;
  printf("%d%d",a,b);
  return 0;
}
```



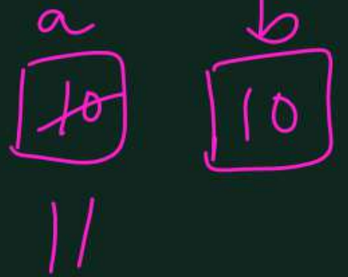
b=++a ⟹ a=a+1 ↓ b=a

```
int a=10;
printf("%d",++a);//11
printf("%d",a--);//11
printf("%d",a);//10
```

```
#include<stdio.h>
int main()
{ int a=10,b;
  b=a++;
  printf("%d%d",a,b);
  return 0;
}
```



b=a++ ⟹ b=a ↓ a=a+1



Important point about ++/--

→ ++/-- operators does not work upon constants

Ex (1)

```
int main()
{
    int x=3, y=2, z;
    z = (2)++;  ⇒ 2 = 2+1 → invalid
                  error
    z = x+y;
    printf("%d", z);
    return 0;
}
```

(2)

```
int main()
{
    const int x=10, y;
    y = x++;        → constant error
    printf("%d", y);
    return 0;
}
```

(3)

```
int main()
{
    int x=10, y=20, z;
    z = (x+y)++;   // error
        (30)
    printf("%d", z);
    return 0;
}
```

Relational Operators

- They are used for the comparison of the two operands. All these operators are binary operators that return true or false values as the result of comparison. (True ⇒ 1, false ⇒ 0)

Symbol	Operator	Description	Syntax
<	Less than	Returns true if the left operand is less than right operand. Else false	a < b
>	Greater than	Returns true if the left operand is greater than right operand. Else false	a > b
<=	Less than or equal to	Returns true if the left operand is less than or equal to the right operand. Else false	a <= b
>=	Greater than or equal to	Returns true if the left operand is greater than or equal to right operand. Else false	a >= b
==	Equal to	Returns true if both the operands are equal.	a == b
!=	Not equal to	Returns true if both the operands are NOT equal.	a != b

```c
// C program to illustrate the relational operators
#include <stdio.h>
int main()
{
        int a = 25, b = 5;
        printf("a < b : %d\n", a < b);
        printf("a > b : %d\n", a > b);
        printf("a <= b: %d\n", a <= b);
        printf("a >= b: %d\n", a >= b);
        printf("a == b: %d\n", a == b);
        printf("a != b : %d\n", a != b);
        return 0;
}
```

False

25 < 5 ⇒ 0

25 > 5 ⇒ T(1)

a = b

a == b

```
Output:

a < b : 0
a > b : 1
a <= b: 0
a >= b: 1
a == b: 0
a != b : 1
```

Logical Operator in C

- Used to combine two or more conditions/constraints or to complement the evaluation of the original condition in consideration.
- Results in Boolean value either **true** or **false**.

non-zero ⇒ True

zero ⇒ false

Symbol	Operator	Description	Syntax
&&	Logical AND	Returns true if both the operands are true.	a && b
||	Logical OR	Returns true if both or any of the operand is true.	a || b
!	Logical NOT	Returns true if the operand is false.	!a

Logical AND

I_1	I_2	O/P
0	0	0
0	1	0
1	0	0
1	1	1

Logical OR

I_1	I_2	O/P
0	0	0
0	1	1
1	0	1
1	1	1

Logical NOT (unary)

I/P	O/P
0	1
1	0

```c
// C program to illustrate the logical operators

#include <stdio.h>

int main()

{

int a = 25, b = 5;

printf("a && b : %d\n", a && b);

printf("a || b : %d\n", a || b);

printf("!a: %d\n", !a);

return 0;

}
```

① a && b

25 && 5 ⇒ T T ⇒ T means ⇒ 1

② a || b ⇒ T

25 || 5

T T

Output:

a && b : 1

a || b : 1

!a: 0

③

!a

!25

!T ⇒ 0

AKTU

B.Tech I-Year

GW GATEWAY CLASSES

PPS: Programming For Problem Solving

UNIT-2

Lecture-2

Topics to be Covered

- Bitwise Operator
- Conditional Operator
- Assignment Operator
- Comma Operator
- sizeof() operator
- **AKTU PYQs**

Video Lectures I Pdf Notes I PYQs

By Dr. Nidhi Parashar Ma'am
- M.Tech Gold Medalist
- Net Qualified

Bitwise Operators in C: Perform bit-level operations on the operands.

- The operands are first converted to bit-level and then calculations such as addition, subtraction, multiplication is performed on the operands for faster processing.

Symbol	Operator	Description	Syntax
&	Bitwise AND	Performs bit-by-bit AND operation and returns the result.	a & b
|	Bitwise OR	Performs bit-by-bit OR operation and returns the result.	a | b
^	Bitwise XOR	Performs bit-by-bit XOR operation and returns the result.	a ^ b
~	Bitwise one's Complement	Flips all the set and unset bits on the number.	~a
<<	Bitwise Left shift	Shifts the number in binary form by one place in the operation and returns the result.	a << b
>>	Bitwise Rightshilft	Shifts the number in binary form by one place in the operation and returns the result.	a >> b

Decimal to Binary / Binary to Decimal NO Conversion

① $(32)_{10} = (100000)_2$

② $(\ )_2 = (?)_{10}$

$\ldots\ldots 2^6\ 2^5\ 2^4\ 2^3\ 2^2\ 2^1\ 2^0$

$\ldots\ldots 128\ 64\ 32\ 16\ 8\ 4\ 2\ 1$

31

2	32	
2	16	0
2	8	0
2	4	0
2	2	0
	1	0
		1

$(100000)_2$

$2^5\ 2^4\ 2^3\ 2^2\ 2^1\ 2^0$

$32\ 16\ 8\ 4\ 2\ 1$

32 → 16 → 8 → 4 → 2 → 1

32	16	8	4	2	1
1	0	0	0	0	0
1	0	0	0	1	1
0	1	0	0	0	0

$(10000)_2$

$2^4\ 2^3\ 2^2\ 2^1\ 2^0$

$1\times2^4 + 0\times2^3 + 0\times2^2 + 0\times2^1 + 0\times2^0$

$= 2^4 = 16$

Bitwise operators

$(25)_{10} = (?)_2 \Rightarrow (11001)_2$

$(5)_{10} = (00101)_2$

16 8 4 2 1
1 1 0 0 1
0 0 1 0 1

① Bitwise AND (&)

(25) & (5)

  1 1 0 0 1
  0 0 1 0 1
  0 0 0 0 1 ⇒ (1)

16 8 4 2 1

② Bitwise OR (|)

(25) | (5)

  1 1 0 0 1
  0 0 1 0 1
  1 1 1 0 1

16 8 4 2 1

16+8+4+1

(29) Any

③ Bitwise XOR (^)

→ It gives 1 if both i/p are different
→ It gives 0 if both i/p are same

25 ⇒ 1 1 0 0 1
5 ⇒ 0 0 1 0 1 (^)
1 1 1 0 0

16 8 4 2 1

(28) Any ✓

④ Bitwise one's comp (~)

↓ unary

~(25)

~(11001) = (00110)$_2$ = (6)

16 8 4 2 1

Bitwise Right shift (>>)

$a = 16$

$a >> 1$

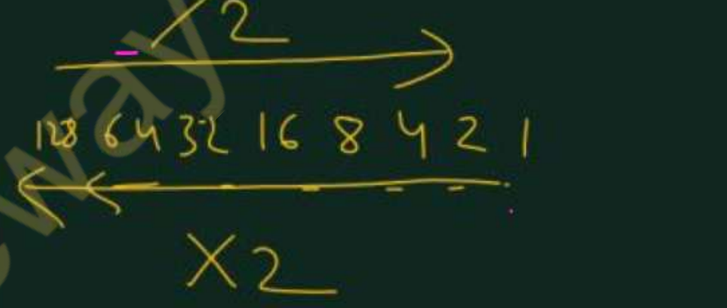
$a = 16 \Rightarrow 0\,1\,0\,0\,0\,0 \Rightarrow 16$

$(a >> 1) = (0\,1\,0\,0\,0) \Rightarrow 8$

$(a >> 2) = (0\,0\,1\,0\,0\,0) \Rightarrow 4$

Note - Right shifting 'a' by x units means

$a >> x \Rightarrow a / 2^x$



Bitwise Left-Shift

$a = 16$

$(a << 1)$

$a \Rightarrow 10000\underline{0}$

$\Downarrow$

32

If you shift 'a' to left by x units then the result will be

Result = $a \times 2^{x}$

$x = 1$

$a \times 2$

$x = 2$

$a \times 4$

⋮

Assignment Operators in C

- Used to assign value to a variable. (=)
- The left side operand of the assignment operator is a variable and the right side operand of the assignment operator is a value / variable / constant / expression.
- The assignment operators can be combined with some other operators in C to provide multiple operations using single operator. These operators are called compound operators.

Difference b/w (=) and (==)

(=)	(==)
→ Assignment operator	→ Relational operator
→ a=b	→ (a==b) → 1 (if a and b are equal), 0 (if unequal)
It assigns value of b to a.	It compares a and b for equality.

a=3 ✓

a=b ✓

a+b=2 ✗

a=b+c; ✓

2=a ✗

Symbol	Operator	Description	Syntax
=	Simple Assignment	Assign the value of the right operand to the left operand.	a = b
+=	Plus and assign	Add the right operand and left operand and assign this value to the left operand.	a += b a = a+b
-=	Minus and assign	Subtract the right operand and left operand and assign this value to the left operand.	a -= b a = a-b
*=	Multiply and assign	Multiply the right operand and left operand and assign this value to the left operand.	a *= b a = a*b
/=	Divide and assign	Divide the left operand with the right operand and assign this value to the left operand.	a /= b a = a/b
%=	Modulus and assign	Assign the remainder in the division of left operand with the right operand to the left operand.	a %= b a = a%b

① $a = 2$

② $a += 2 \Rightarrow a = a + 2$

$a += 2$

③ $a -= 2 \Rightarrow a = a - 2$

④ $a += 1 \Rightarrow a = a + 1$

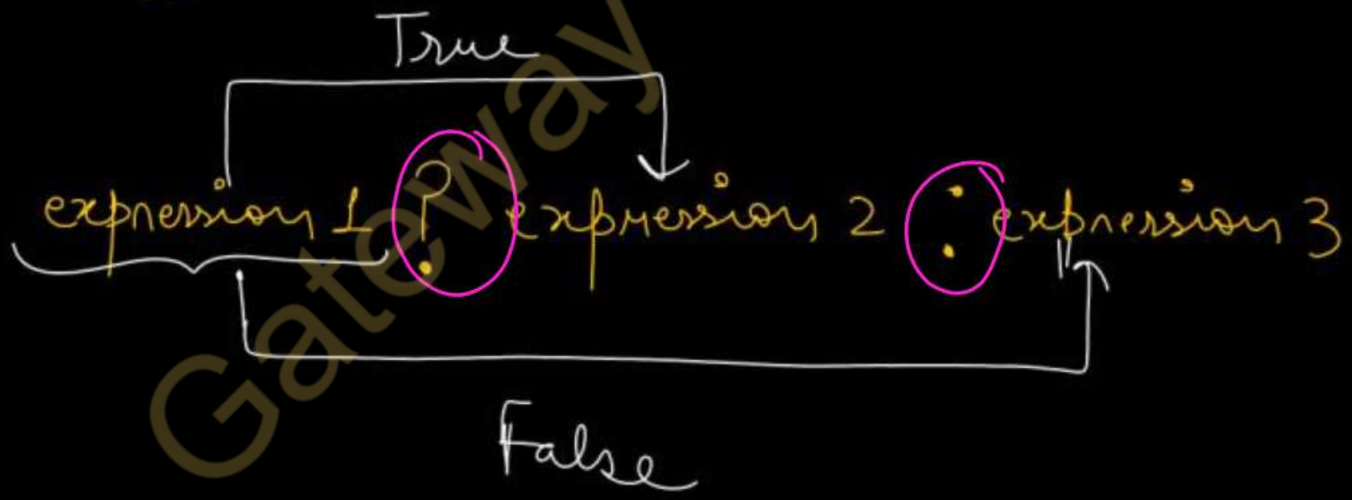
$++a$ / $a++$

$+=$
$-=$
$*=$
$/=$
$\%=$

compound assignment operators

Conditional Operator (? :) (Imp)

- The conditional operator is the only ternary operator in C.
- Here, Expression1 is the condition to be evaluated. If the condition(Expression1) is *True* then we will execute and return the result of Expression2 otherwise we will execute and return the result of Expression3.
- We may replace the use of if.. else statements with conditional operators.



Write a program to print larger number between two numbers using conditional operator

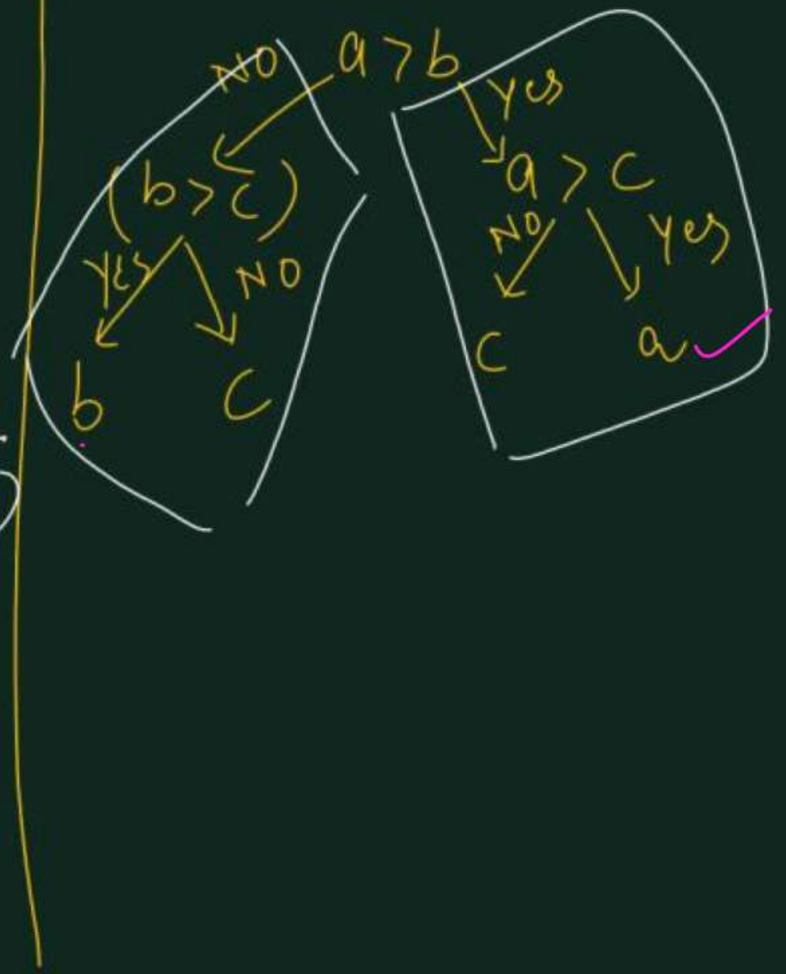
a=10, b=20

```
#include <stdio.h>
int main()
{
    int a, b, big;
    printf("Enter two numbers to be compared");
    scanf("%d%d", &a, &b);
    big = (a>b)? a : b;
    printf("The larger number is = %d", big);
    return 0;
}
```

V. Imp Program to print the largest number among three Numbers using Conditional operator (AKTU)

```
#include<stdio.h>
int main()
{
    int a, b, c, big;
    printf("Enter three numbers to be compared");
    scanf("%d%d%d", &a, &b, &c);
    big = (a>b)?(a>c?a:c):(b>c?b:c);
    printf("The largest number = %d", big);
    return 0;
}
```

a b c



sizeof Operator: It is unary operator which can be used to compute the size of its operand(variable or datatype).

```
#include <stdio.h>
int main() {
    int myInt;
    float myFloat;
    double myDouble;
    char myChar;
    printf("%lu\n", sizeof(myInt));
    printf("%lu\n", sizeof(myFloat));
    printf("%lu\n", sizeof(myDouble));
    printf("%lu\n", sizeof(myChar));
    return 0;
}
```

int myInt; ⇒ 2 byte
float myFloat; ⇒ 4 byte
double myDouble; ⇒ 8 byte
char myChar; ⇒ 1 byte

// 2
// 4
// 8
// 1

operand

sizeof (int) ⇒ 2

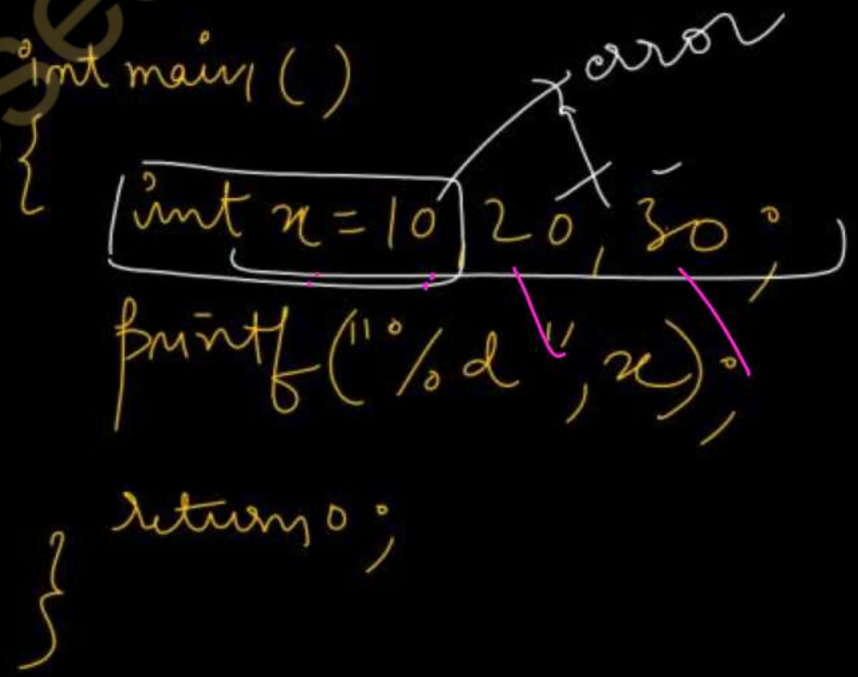
sizeof (20) ⇒ 2

int a = 30;
sizeof (a); ⇒ 2

Comma as an Operator

- A comma operator in C s a binary operator.
- It evaluates the first operand & discards the result, evaluates the second operand & returns the value as a result.

```
#include <stdio.h>
int main()
{
        int x = 10;
        int y = (15, 20);
        printf("%d", y); // It will print 20
        return 0;
}
```



AKTU PYQs

1. Define conditional operator with example. (AKTU 2023-24)

2. Explain different types of bitwise operators used in C with suitable examples. Find the value of following expressions:

i) 10 >>2 ii) 20 << 2 iii) 25 & 30 iv) 25 | 30 (AKTU 2022-23)

3. Explain bitwise operator with example. (AKTU 2021-22)

4. Write a program to find largest among three numbers using conditional operator. (AKTU 2021-22)

5. Explain logical, unary and bitwise operators in detail. (AKTU 2021-22)

AKTU

B.Tech I-Year

GW GATEWAY CLASSES

PPS: Programming For Problem Solving

UNIT-2

Lecture-3

Topics to be Covered

- Operator precedence and associativity
- Mixed Operands
- Type conversion and type casting
- AKTU PYQs

Video Lectures I Pdf Notes I PYQs

By Dr. Nidhi Parashar Ma'am
- M.Tech Gold Medalist
- Net Qualified

Precedence and Associativity (AKTU)

The precedence of an operator gives the order in which operators are applied in expressions:

- The highest precedence operator is applied first, followed by the next highest, and so on.
- For example * has high precedence than +.

Ex → `x = 100 + 30 * 2 / 3 - 1;`

⇒ `100 + 60 / 3 - 1`

⇒ `100 + 20 - 1`

⇒ `120 - 1`

⇒ `119`

⇒ `x = 119`

* /
+ -

precedence ↑

associativity: Left to Right

Associativity rules tell that how the operator of same precedence is grouped and how the expression will be evaluated. For example arithmetic operator is left associative but assignment operator is right associative.

`a = b = 3`

Precedence	Operator	Description	Associativity
Highest priority 1	()	Parentheses (function call)	Left-to-Right
same precedence	[]	Array Subscript (Square Brackets)	
	.	Dot Operator	
	->	Structure Pointer Operator	
	++ , --	Postfix increment, decrement	
same precedence 2	++ / --	Prefix increment, decrement	Right-to-Left
	+ / -	Unary plus, minus	
	! , ~	Logical NOT, Bitwise complement	
	(type)	Cast Operator	
	*	Dereference Operator	
	&	Addressof Operator	
	sizeof	Determine size in bytes	

Precedence	Operator	Description	Associativity
3	*,/,%	Multiplication, division, modulus	Left-to-Right
4	+/-	Addition, subtraction	Left-to-Right
5	<< , >>	Bitwise shift left, Bitwise shift right	Left-to-Right
6	< , <=	Relational less than, less than or equal to	Left-to-Right
	> , >=	Relational greater than, greater than or equal to	
7	== , !=	Relational is equal to, is not equal to	Left-to-Right
8	&	Bitwise AND	Left-to-Right

Precedence	Operator	Description	Associativity
9	^	Bitwise exclusive OR	Left-to-Right
10	|	Bitwise inclusive OR	Left-to-Right
11	&&	Logical AND	Left-to-Right
12	||	Logical OR	Left-to-Right
13	?:	Ternary conditional	Right-to-Left

Precedence	Operator	Description	Associativity
14	=	Assignment	Right-to-Left
	+= , -=	Addition, subtraction assignment	
	*= , /=	Multiplication, division assignment	
	%= , &=	Modulus, bitwise AND assignment	
	^= , |=	Bitwise exclusive, inclusive OR assignment	
	<<=, >>=	Bitwise shift left, right assignment	
15	,	comma (expression separator)	Left-to-Right

①

```
int main()
{
int x = 10;
x = 100 + 200 / 10 - 3 * 1;
printf("%d", x);  // 117
return 0;
}
```

$x = 100 + 200/10 - 3*1$

+ ②	$x = 100 + 20 - 3*1$
− ②	$= 100 + 20 - 3$
* ①	$= 120 - 3$
/ ①	$x = 117$ ✓
= ③	

```
int main()
{
int x = 10;
int y;
y = 12,15;
printf("%d", y);  // 12
return 0;
}
```

$y = (12, 15)$ → ignore

$y = 12, 15$

As the precedence of = is higher than ,

```
int main()
{
int a = 10, b = 20, c = 30,x;
x= (c > b > a);
printf("%d", x );  // 0
return 0;
 }
```

$x = (c > b > a);$

$= 30 > 20 > 10$

$= 1 > 10$

$= 0$

Mixed-Mode Arithmetic (Mixed operands) (AKTU)

- Mixing integers and floating-point numbers in a basic arithmetic operation
- Converts the integers automatically to floating point.
- If i and j are integers, (i + j) * 2.0 adds them as an integer, then converts the sum to floating point for the multiplication.
- Thus, 3.0 + i + j converts i to floating point, then adds 3.0, then converts j to floating point and adds
- that. You can specify a different order using parentheses: 3.0 + (i + j) adds i and j first and then adds that sum (converted to floating point) to 3.0.

Examples:

- **1 + 2.5** is **3.5**
- **1/2.0** is **0.5**
- **2.0/8** is **0.25**

1 + 2.5
↓
1.0 + 2.5 ⇒ 3.5

1 / 2.0
1.0 / 2.0 = 0.5

Type Casting and Conversion in C (Important)

If the expression contains two or more different datatype values then they must be converted to the single datatype of destination datatype.

For example: Multiplication of an integer value with the float value and storing the result into a float variable. In this case, the integer value must be converted to float value so that the final result is a float datatype value.

```
int a = 2;
float b = 3, c;
c = a * b;
```

has to be converted to float so that correct float value can be stored in c

In a c programming language, the data conversion is performed in two different methods as follows...

1. Type Conversion
2. Type Casting

Type Conversion

- Process of converting a data value from one data type to another data type automatically by the compiler.
- Also called implicit type conversion. When compiler converts implicitly, there may be a data loss.

For example, in c programming language, when we assign an integer value to a float variable the integer value automatically gets converted to float value by adding decimal value 0. And when a float value is assigned to an integer variable the float value automatically gets converted to an integer value by removing the decimal value.

Implicit (automatic) Type Conversion

Type Conversion
- Implicit → automatic
- Explicit → Type Casting

(1)

```
int main()
{
    int x = 5, y = 2, z;
    z = x / y;
    printf("%d", z); // 2
    return 0;
}
```

int / int ⇒ int 5/2 ⇒ (2)

5/2 = 2.5

(2)

```
int main()
{
    int z = 5, y = 2;
    float z;
    z = x / y;
    printf("%f", z); // 2.000000
    return 0;
}
```

Implicit Type conversions

(3)

```
main()
{
    float x = 2.15;
    int y = x;
}
```

2

y = 2

(4)

```
int x = 3;
float y = x;
```

y = 3.00

(5)

```
int a = 2.5 * 3;
```

float

= 2.5 * 3.0

= 7.5

←

a = 7

Example of Implicit Type Conversion

```
#include<stdio.h>
 void main()
{
    int i = 95 ;
    float x = 90.99 ;
    char ch = 'A' ;
    i = x ;
    printf("i value is %d\n",i);
    x = i ;
    printf("x value is %f\n",x);
    i = ch ;
    printf("i value is %d\n",i);
}
```

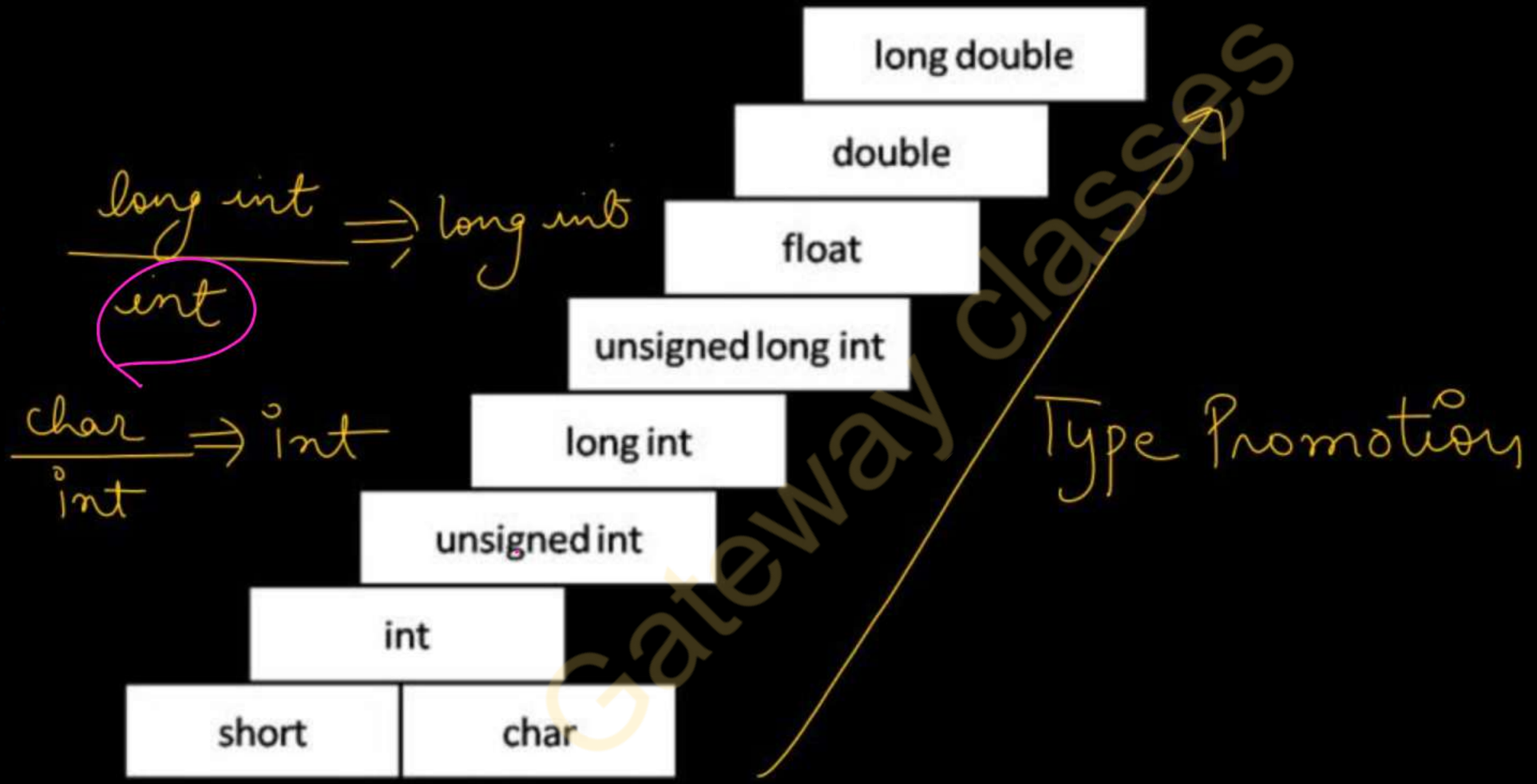
Ascii Value of 'A' is 65

90.00

OUTPUT

```
i value is 90
x value is 90.000000
i value is 65
```

Implicit Type Conversion



Typecasting (Explicit Type Conversion)

- Also called an explicit type conversion.
- When compiler converts implicitly, there may be a data loss. In such a case, we convert the data from one data type to another data type using explicit type conversion(No data loss)
- We convert the data from one data type to another data type using explicit type conversion using the unary cast operator. The general syntax of typecasting is as follows.

Cast operator :→ (target Data type) value

```
int a=5, b=2, c;
c = a/b;
```

(2)

5/2 ⇒ 2

```
int a=5, b=2;
float c;
c = a/b;
```

5/2 ⇒ 2

c = 2.00

```
int a=5;
float b=2, c;
c = a/b
c = 5/2.0
```

5.0/2.0

c = 2.5 ✓

Type Casting

```
int a=5, b=2;
float c;
c=(float)a/b;
```
$\Rightarrow c = 5.0/2 \Rightarrow 2.5$

OR

```
c= a/(float)b;
```
$\Rightarrow c = 5/2.0 \Rightarrow 2.5$

OR

```
c=(float)a/(float)b;
```
$\Rightarrow c = 5.0/2.0 \Rightarrow 2.5$

Example of Type Casting

```
#include<stdio.h>
int main(){
    int a, b, c ;
    float avg ;
    printf("Enter any three integer values : ");
    scanf("%d%d%d",&a,&b,&c);
    avg = (a + b + c)/3;  // 13.00
    printf("avg=%f",avg);
    avg = (float)(a + b + c) / 3 ;  // 13.33
    printf("avg after casting =%f ",avg);
    return 0;
}
```

Suppose a = 10, b = 10, c = 20

40/3 ⇒

implicit type conversion → data loss

typecasting → No data loss

avg = (float)(a+b+c)/3

= (float) 40 / 3

= 40.00... / 3

40.000 / 3.000

avg = 13.333...

AKTU PYQs

1. Explain concept of type conversion and type casting using programs. (AKTU 2023-24)(AKTU 2021-22)
2. Illustrate concept of operator precedence and associativity with example. (AKTU 2023-24) (AKTU 2021-22)
3. Differentiate between implicit and explicit type conversion. (AKTU 2022-23)
4. What is meant by type conversion? Explain about implicit and explicit type conversion.(AKTU 2021-22)(AKTU 2019-20)
5. Differentiate between type conversion and type casting.

(AKTU 2021-22)

6. What do you mean by operands? Illustrate the concept of operator precedence and associativity of all operators with example. (AKTU 2021-22) (AKTU 2020-21)(AKTU 2019-20)
7. Explain different types of operators in detail. Which concept makes the difference between operators when precedence is same? (AKTU 2021-22)

AKTU

B.Tech I-Year

GATEWAY CLASSES

PPS: Programming For Problem Solving

UNIT-2

Lecture-4

Topics to be Covered

- Output Questions based on operators
- AKTU PYQ(output questions on operators)
- **AKTU PYQs**

Video Lectures I Pdf Notes I PYQs

By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

Output Questions

```
#include <stdio.h>
int main()
{
    int i = 5, j = 10, k = 15;
    printf("%d ", sizeof(k /= i + j));
    printf("%d", k);
    return 0;
}
```

int

Expression is not evaluated in sizeof()

output:

4 15

```
#include <stdio.h>
int main()
{
printf("%d", sizeof(printf("hello")));
return 0;
}
```

5 ⇒ int

output:

4

```
#include <stdio.h>
int main(void)
{
    int y;
    int x = 11;
    y = sizeof(x++);
    printf("%d %d", y, x);
    return (0);
}
```

4 ✓

=

4 11

x++ will not be evaluated

so the value of x remains same

Output Questions

```
#include <stdio.h>
int main()
{
   int a = 1;
   int b = 1;
   int c = a || --b;
   int d = a-- && --b;
   printf("a = %d, b = %d, c = %d, d = %d", a, b, c, d);
   return 0;
}
```

① c = a || --b;

(T) || --b ⇒ short circuit evaluation

True (1)

// a = 1, b = 1, c = 1

② d = a-- && --b;

1 && 0

0

output :→ 0 0 1 0

```
main()
{
  int x=0, y=-18, z;
  z = x && (++y);
  printf("%d %d", y, z);
}
```

false

not evaluated due to short circuit evaluation.

output: -18 0

Note

Any non-zero value is "True"

Zero is "false"

Output Questions

```
#include <stdio.h>
int main()
{
   printf("%d", 1 << 2 + 3 << 4);
   return 0;
}
```

output : 512

. . . 32 16 8 4 2 1

$x << n$

$x * 2^n$

Note → Precedance of + is higher than bitwise left shift (<<) → L-R associative

1 << 2+3 << 4

⇒ 1 << 5 << 4

1×2^5

= 32 << 4

$= 32 \times 2^4$ ⇒ 32×16

= 512

```
#include <stdio.h>
int main()
{
   printf("%d", 35 >> 4);
   return 0;
}
```

output : 2

$x >> n$

$x / 2^n$

35 >> 4

$= 35 / 2^4$

$= 35 / 16$

= 2 ✓

Output Questions

```
#include<stdio.h>
int main()
{
  int a = 2,b = 5;
  a = a^b; // a = 7, b = 5
  b = b^a; // a = 7, b = 2
  printf("%d %d",a,b);
  return 0;
}
```

7 2

output:

7 2

^ → Bitwise XOR operation

a = a^b →

```
  0 0 1 0
^ 0 1 0 1
---------
  0 1 1 1
```

→ 2

a = 7

b = b^a ⇒

```
  0 1 0 1
^ 0 1 1 1
---------
  0 0 1 0
```

⇒ 2

b = 2

```
  8 4 2 1
2 → 0 0 1 0
5 → 0 1 0 1
```

AKTU 2021-22

```
#include <stdio.h>
int main()
{
int a=10;
printf("number=%d\n"+1,a);
printf("value=%d"+3,a);
return 0;
}
```

printf("number=%d\n"+1, a)

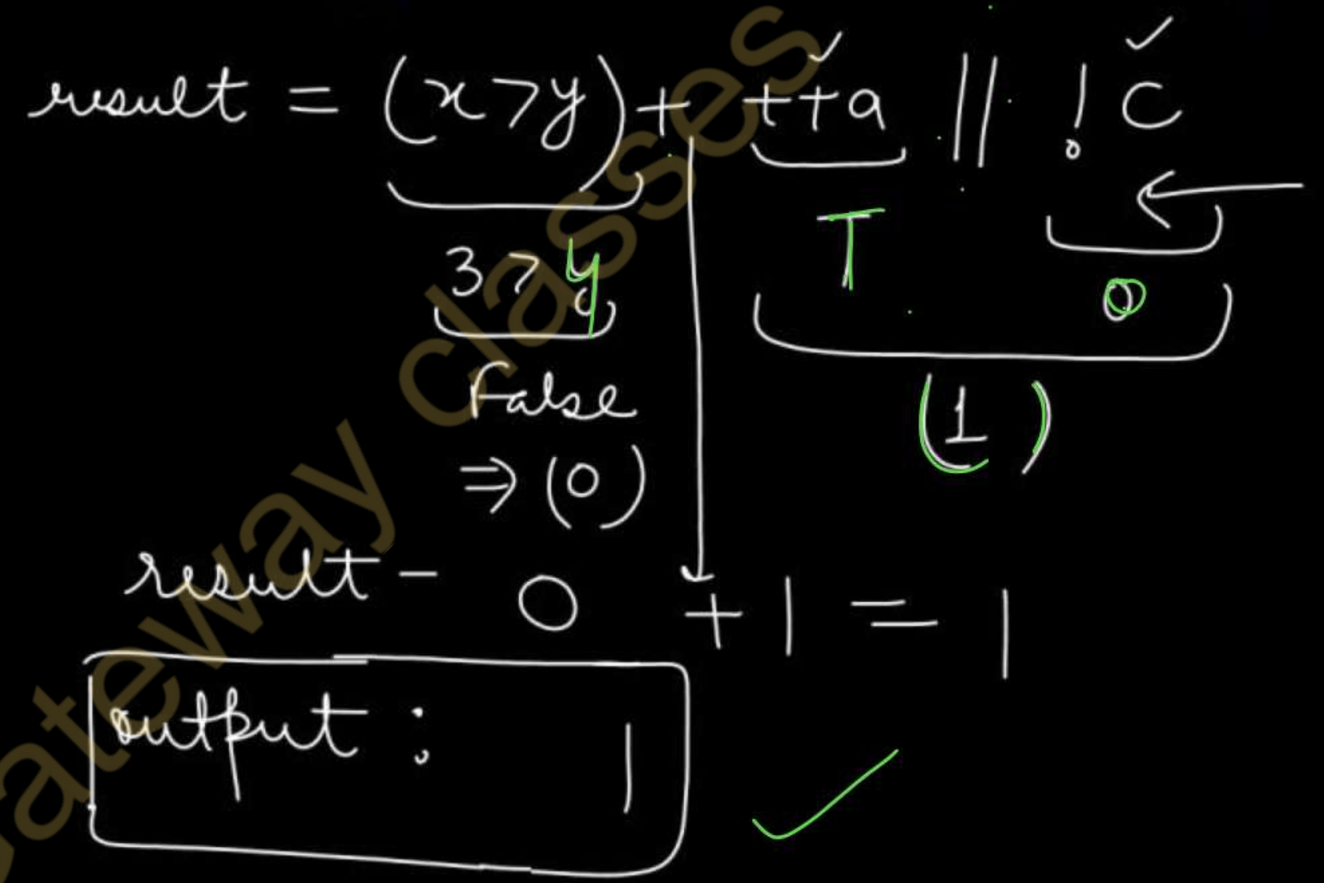
output :- umber = 10

ue = 10

AKTU 2021-22

```
#include <stdio.h>
int main()
{
int x=3,y=4,a=6,c=7,result;
result=(x>y)+ ++a ||!c;
printf("%d", result);
return 0;
}
```

Note: Precedence of pre increment and ! is same and associativity is right to left



```c
#include<stdio.h>
int main()
{
    int x, y;
    x = 5;
    y = x++ / 2;  // 2
    printf("%d%d", x,y);
    return 0;
}
```

6 2

$y = x++ / 2$

$y = x/2$

$x = x+1$

output :

6 2

```c
#include<stdio.h>
int main()
{
    int a=4,b,c;
    b = --a;  // a=3, b=3
    c = a--;  // c=3, a=2
    printf("%d %d %d", a, b, c);

    return 0;
}
```

output : 2 3 3

```c
#include<stdio.h>
int main()
{
int a=5;
printf("%d", ++a++);
return 0;
}
```

↓

"Invalid"

Error – Lvalue Required

```c
#include<stdio.h>
int main()
{
    printf("%d ", + +9);
    printf("%d ", + -9);
    printf("%d ", - +9);
    printf("%d ", - -9);
    return 0;
}
```

Output: 9 -9 -9 9

```c
#include<stdio.h>
int main()
{
  int a=10, b=10,c;
  c= a++ + ++b;
  printf("%d %d %d",a,b,c);
  return 0;
}
```

c = a++ + ++b;

10 + 11

c = 21

a = 11

b = 11

Output : 11 11 21

AKTU

B.Tech I-Year

GATEWAY CLASSES

PPS: Programming For Problem Solving

UNIT-2

Lecture-5

Topics to be Covered

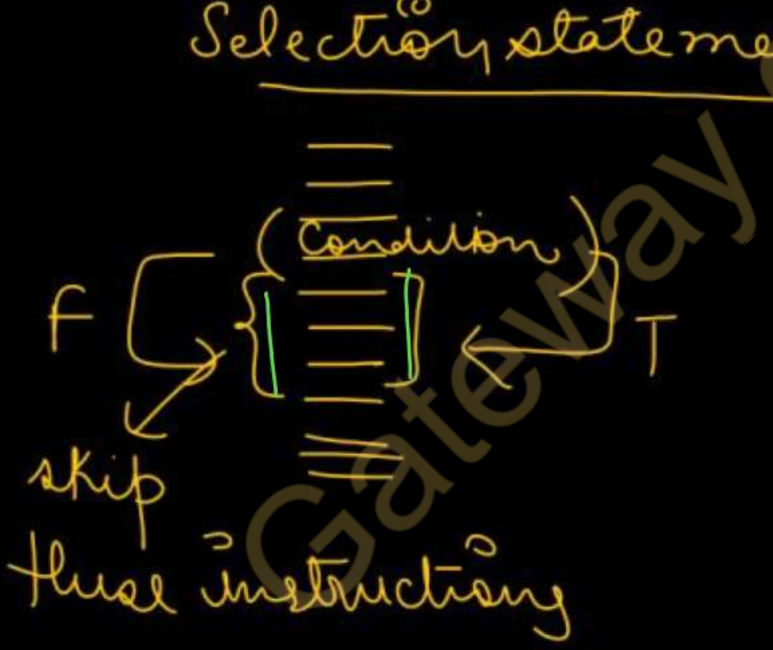
- Control Statements
  - if statement
  - if-else statement
  - Program to check even/odd number
  - Program to print absolute value
  - Program to check for divisibility
  - Program to print bigger number between two numbers
  - Program to check character is vowel or consonant
- AKTU PYQs

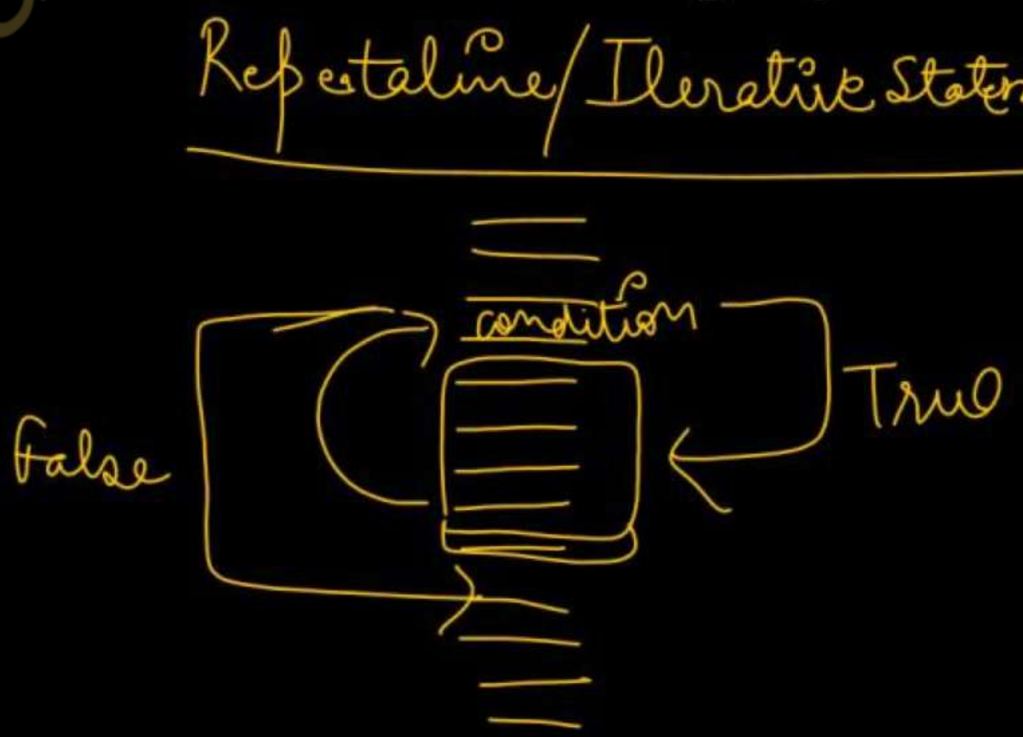
Video Lectures I Pdf Notes I PYQs

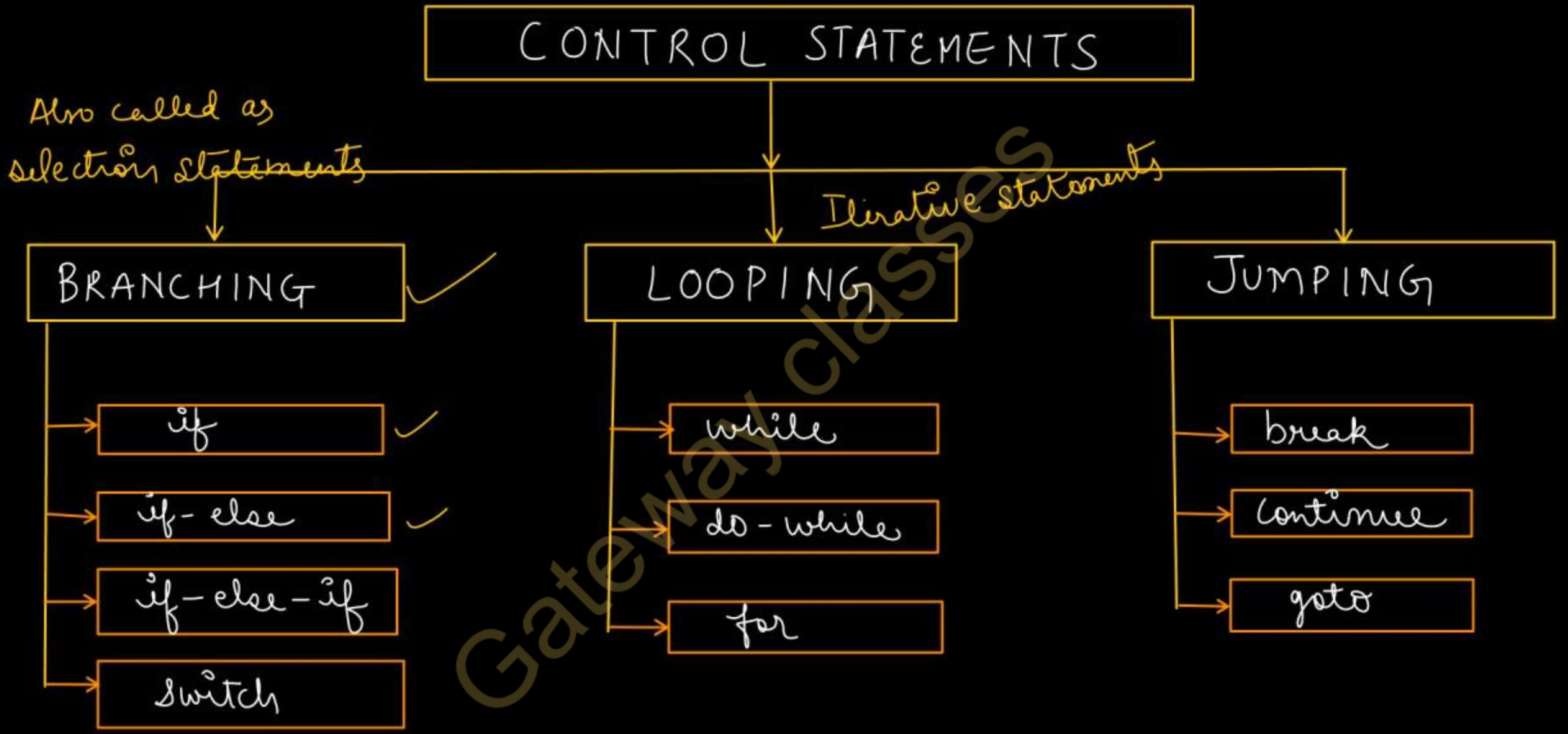
By Dr. Nidhi Parashar Ma'am
- M.Tech Gold Medalist
- Net Qualified

Control Statements

In C, programs are executed sequentially in the order of which they appear. Sometimes we need to execute a certain part of the program or we may want to execute the same part more than once. Control statements enable us to specify the order in which the various instructions in the program are to be executed. Control statements are classified in the following ways:





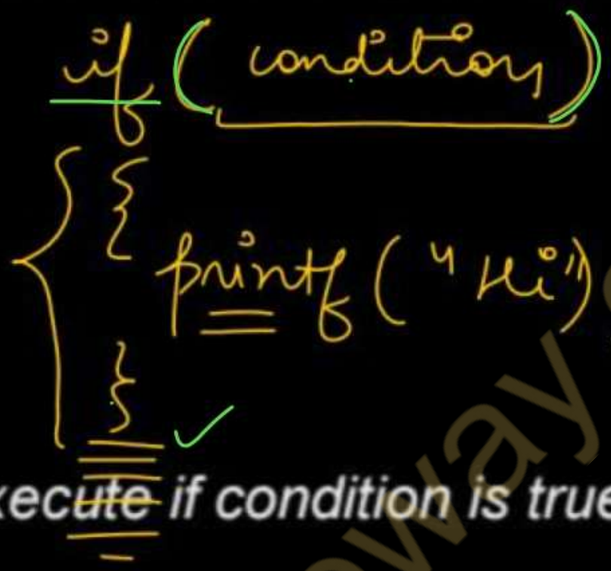


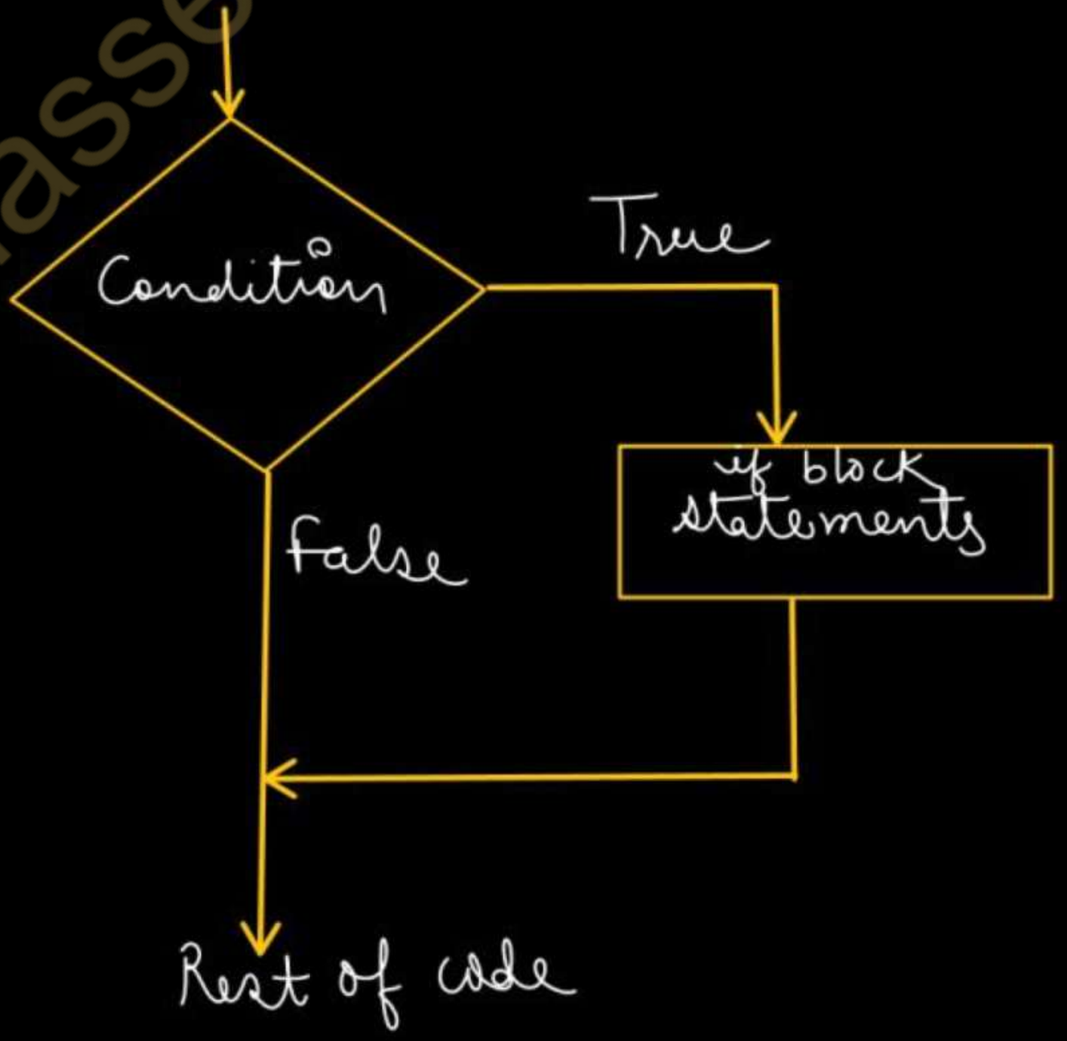
1. if statement

- It is used to decide whether a certain statement or block of statements will be executed or not
- if a certain condition is true then a block of statements is executed otherwise not.

Syntax:

```
if(condition)
{
        // Statements to execute if condition is true
}
//Rest of the code
```





```
#include<stdio.h>
int main()
{ int x=10;
  if(x==10)
  {
    printf("Hello\n");
    printf("world\n");
  }
  printf("India");
  return 0;
}
```

if-block

output → Hello
world
India

```
int main()
{ int x=10;
  if(x==10)
    printf("Hello");
  printf("world");
  printf("India");
  return 0;
}
```

printf("Hello"); ⇒ body of if

printf("world"); ⇒ not a part of if block

output: Hello
world
India

```
#include<stdio.h>
int main()
{ int x=5;
  if(x>10)
    printf("Hello");
  printf("World");
}
```

x>10 (false)

5>10

false

output:

World

```
#include<stdio.h>
int main()
{ int x=5, y=0;
  if(x)
    printf("Hello");
  printf("World");
  if(y)
    printf("India");
  return 0;
}
```

x's value is 5 so it evaluates to be true

it evaluates to be false

output:

Hello

World

if-else Statement:

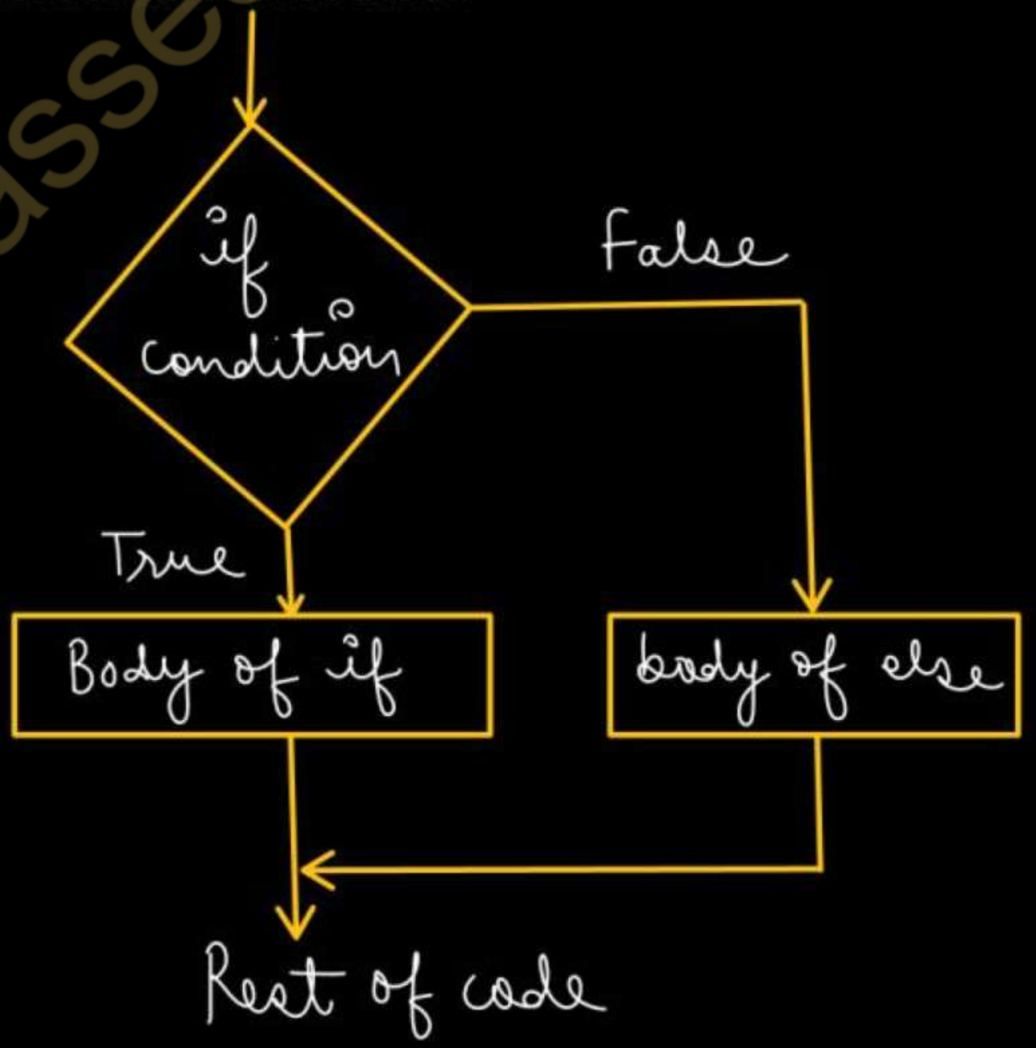
- when the *if condition* evaluates to true , the statements inside if block are executed.
- If the condition evaluates to false, statements inside else block are executed.

Syntax:

```
if (condition)
{
    Statements..;
}
else
{
    Statements..;
}
Rest of code.....
```

body of if

body of else



```
int x=10;   5
if (x==10)  (T)
{
    printf("Hello");
    printf("world");
}
else
{
    printf("India");
    printf("USA");
}
```

x=10 ✓ x=5 ✗

```
int x = 10;
if (x == 20)
    printf("Hello");
    printf("world");
else
{
    printf("India");
}
```

→ false

body of if

⇐ Rest of code

→ Misplaced-else error

```
int x = 10;
if (x == 20)
{
    printf("Hello");
    printf("world");
}
else
    printf("India");
printf("USA");
```

false

output : India USA

No error !

Rest of code

1. Write a program to check whether the given number is even or odd.

```
#include <stdio.h>
int main()
{
  int a;
  printf("Enter the number");
  scanf("%d",&a);
  if(a%2==0)
  {
    printf("%d is Even",a);
  }
  else
  {
    printf("%d is odd",a);
  }
  return 0;
}
```

a = 8

8 is even

a = 25

25 is odd

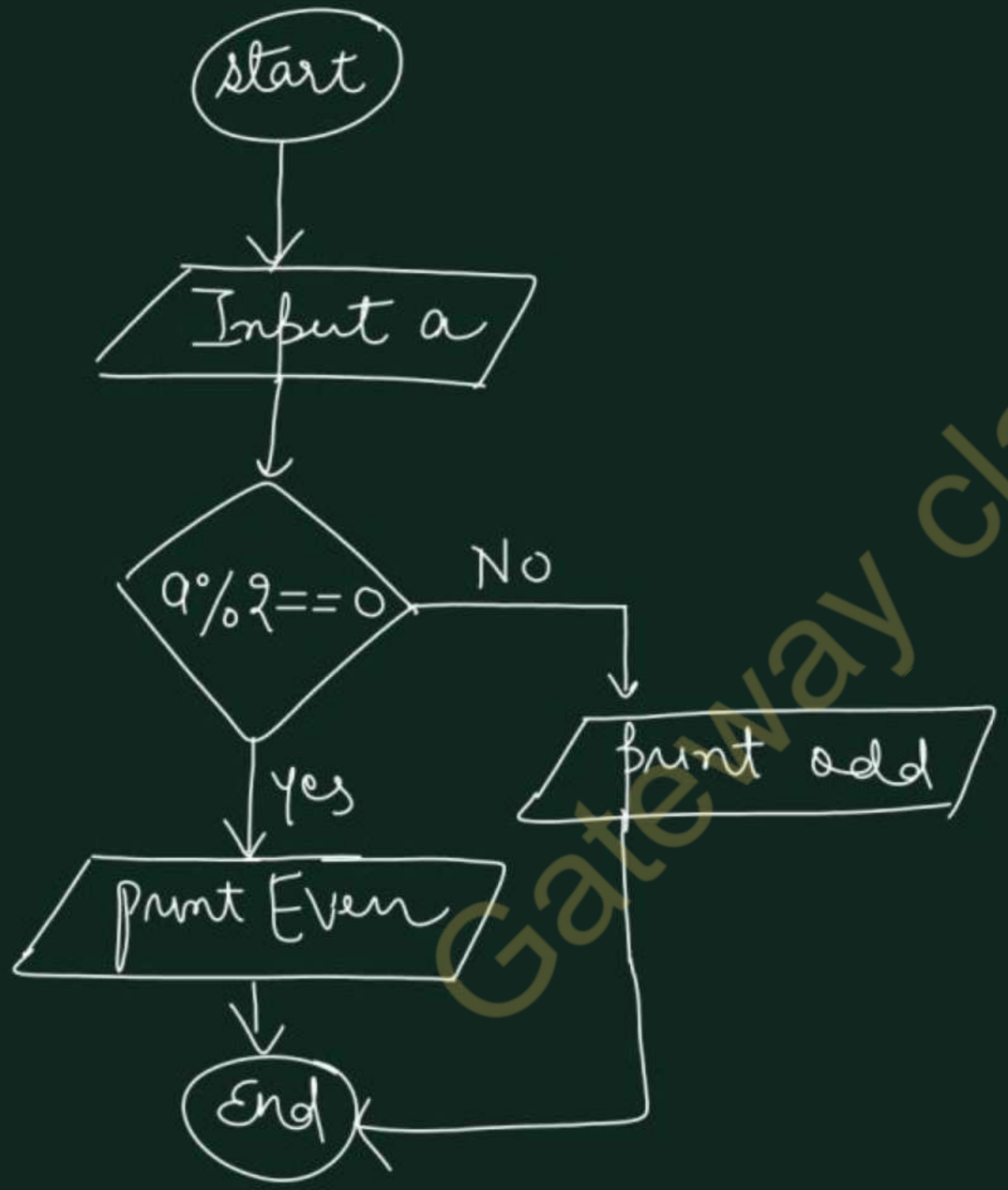
8, 16, 25

	2)8(4	2)16(8	2)25(12
	8	16	24
Remainder	0	0	1

```
x=(a%2==0)?1:0;
if(x==1
```

if (a%2 == 0)

Remainder — compare with 0



Algo

1. start
2. Input a
3. check if (a%2==0)
   print even
   otherwise
   print odd.
4. End.

2. Write a program to print the bigger number between two numbers.

```
#include<stdio.h>
int main()
{ int a, b;
 printf("Enter two numbers");
 scanf("%d%d", &a, &b);
 if(a>b)
   { printf("%d is bigger", a);
   }
 else{ printf("%d is bigger", b);
   }
} return 0;
```

Ex ① a = 10, b = 20

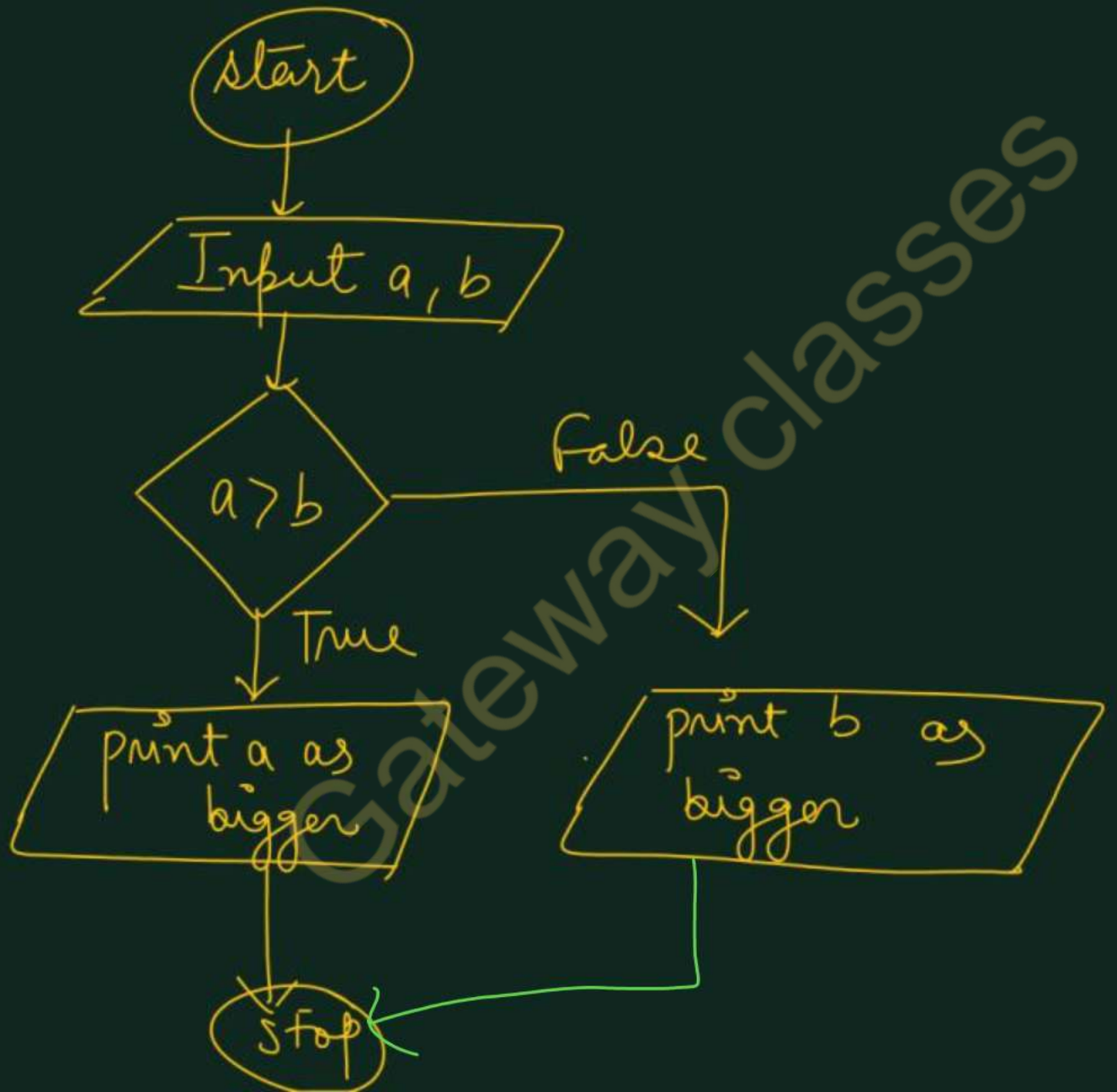
10 > 20 ⇒ False

20 is bigger

② a = 20, b = 10

20 > 10

20 is bigger



3. Write a program to print the absolute value of a number.

absolute value → only magnitude

```
#include<stdio.h>
int main()
{ int a;
  printf("Enter the number");
  scanf("%d",&a);
  if(a<0)
    {  a=a*-1;
       printf("Absolute value = %d",a);
    }
  else { printf("The absolute value is=%d",a);
       }
} return 0;
```

a = -10

a = -10 X -1

a = 10 ✓

AKTU

B.Tech I-Year

GW GATEWAY CLASSES

PPS: Programming For Problem Solving

UNIT-1

Lecture-6

Topics to be Covered

- Program to check for divisibility
- Program to print bigger number between two numbers
- Program to check character is vowel or consonant
- if-else ladder
- WAP to print grades as per following criteria for obtained percentage of marks M out of 100
- WAP to Program to find largest of the three numbers by using &&

By Dr. Nidhi Parashar Ma'am
- M.Tech Gold Medalist
- Net Qualified

Video Lectures I Pdf Notes I Hand Written Notes I DPP

Write a program to check if a given number is divisible by both 5 and 7 or not.

```
#include <stdio.h>
int main()
{
    int n;
    printf("Enter the number");
    scanf("%d", &n);
    if( (n%5==0) && (n%7==0))
    {
        printf("%d is divisible by both 5 and 7", n);
    }
    else
    {
        printf("%d is not divisible by 5 and 7", n);
    }
    return 0;
}
```

(n%5==0) && (n%7==0)

n — check for equality

n%5 → Remainder

Write a program to print 'Yes' if a given number is divisible by either 2 or 3. Otherwise Print 'No'.

```
#include<stdio.h>
int main()
{ int n;
printf("Enter the number");
scanf("%d",&n);
if((n%2==0)||(n%3==0))
  { printf("YES");
  }
else
  { printf("NO");
  }
return 0;
}
```

$(n\%2==0) \;||\; (n\%3==0)$

T T

$n = 9 \rightarrow (9\%2==0) \;||\; (9\%3==0)$

False True

T

"yes" is printed.

$n=6 \Rightarrow$ YES

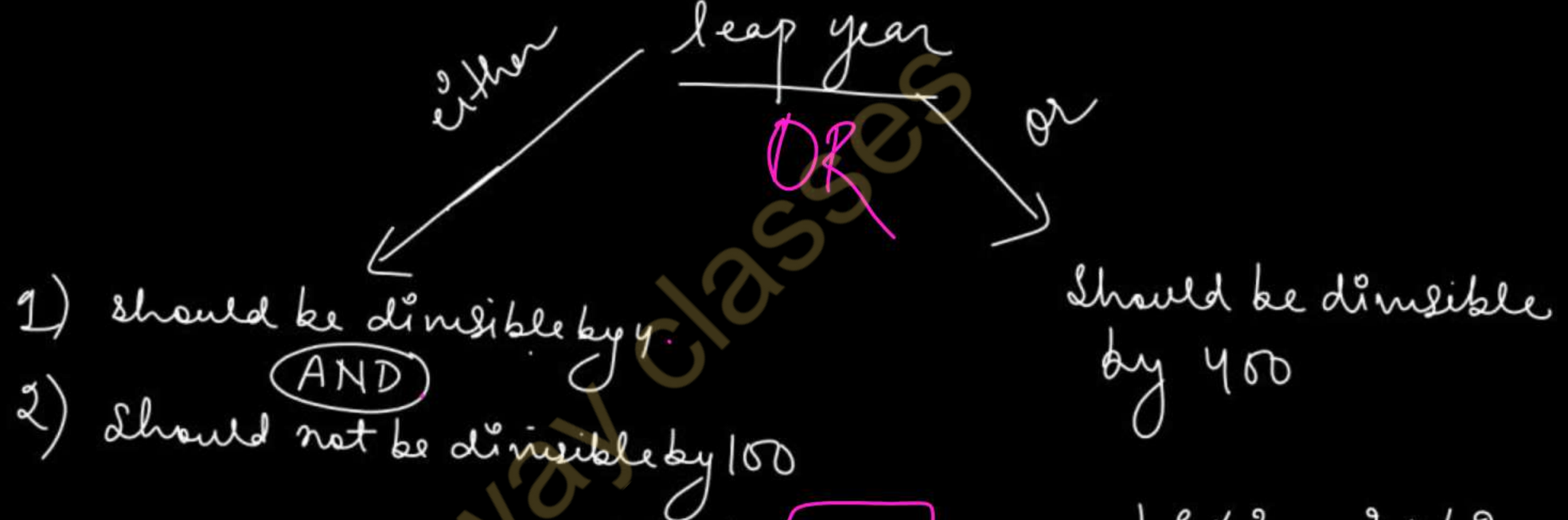
$n=7 \Rightarrow$ NO

Write a program to check if the entered character is vowel or consonant.

```
#include<stdio.h>
int main()
{ char ch;
  printf("Enter the character");
  scanf("%c",&ch);
if((ch=='a')||(ch=='e')||(ch=='i')||(ch=='o')||(ch=='u')||(ch=='A')||
                (ch=='E')||(ch=='I')||(ch=='O')||(ch=='U'))
{
  printf("%c is vowel",ch);
}
else{
  printf("%c is consonant",ch);
}
return 0;
}
```

a e i o u
A E I O U

Write a program to check whether a given year is leap year or not (V. Imp)



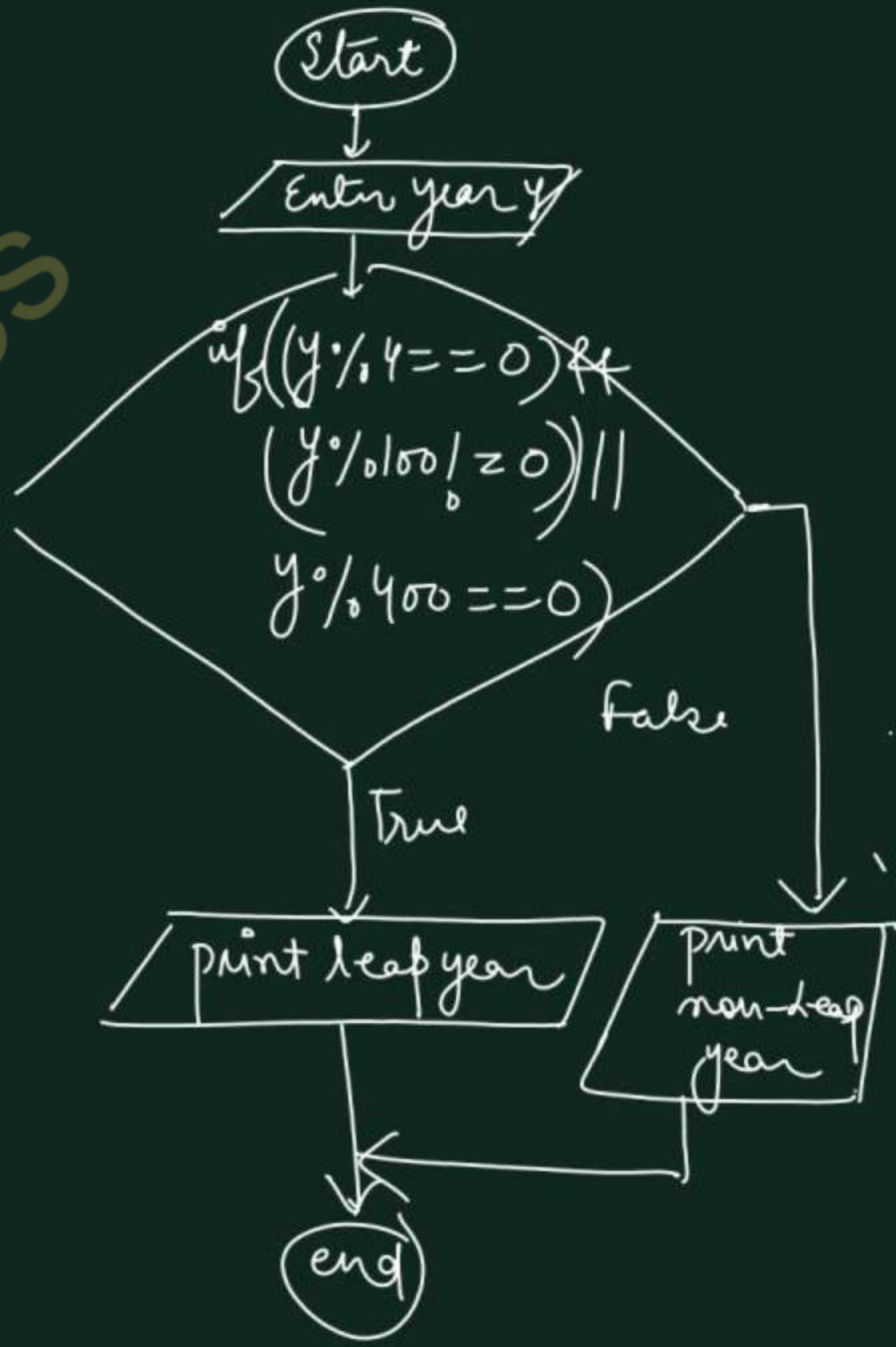
Ex 1: 1900 → divisible by 4 ✓
→ divisible by 100 ✓
↓
not divisible by 400
⇒ Non-leap year

Ex-2: 2000
4 ✓
100 ✓
400 ✓
⇒ Leap year

Ex3: 2012
- divisible by 4
- Not divisible by 100
⇒ Leap year

```
#include<stdio.h>
int main()
{ int y;
 printf("Enter the year");
 scanf("%d",&y);
if( ((y%4==0)&&(y%100!=0)) || (y%400==0))
  {
    printf("%d is a leap year",y);
  }
  else
   { printf("%d is a Non-leap year",y);
   }
    return 0;
  }
```

condition 1 | condition 2

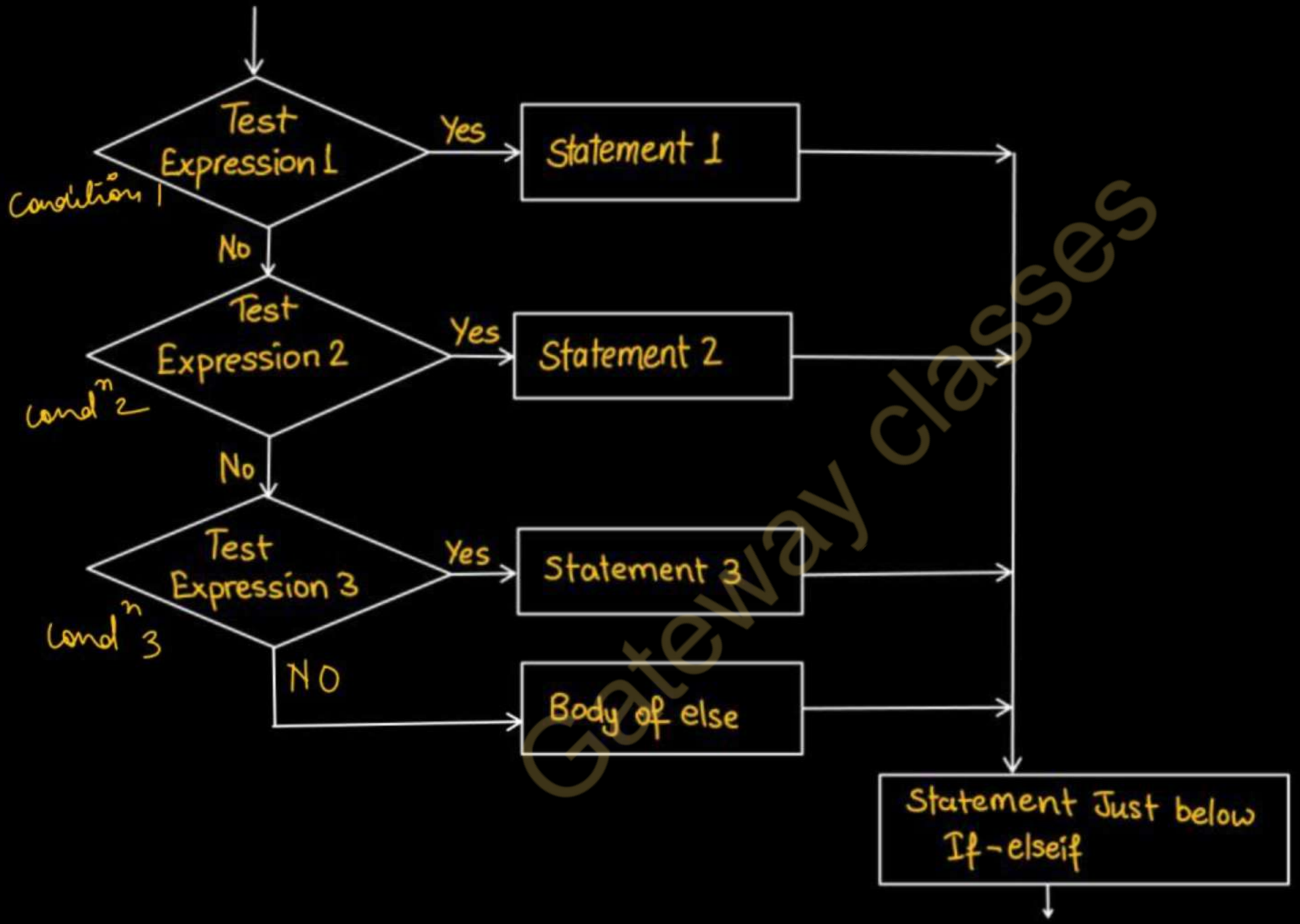


Syntax

if-else-if (if-else ladder)

```
if (condition1)
{
    Statement 1;
}
else if (condition2)
{
    Statement 2;
}
else if (condition3)
{
    Statement 3;
}
else
{
    body of else;
}
```

- First of all condition1 is tested and if it is true then statement1 will be executed and control comes out of whole if else ladder.
- If condition1 is false then only condition2 is tested. Control will keep on flowing downward.
- If none of the conditional is true. The last else will be executed.



(AKTU 2022-23)

```
#include <stdio.h>
int main()
{
    int a = -10, b = 20;
    if(a > 0 && b < 0)
        a++;
    else if(a < 0 && b < 0)
        a--;
    else if(a < 0 && b > 0)
        b--;
    else
        b--;
printf("%d\n",a + b);
return 0;
}
```

① a > 0 && b < 0
-10 > 0 && ___
False

② a < 0 && b < 0
-10 < 0 && 20 < 0
T F

③ a < 0 && b > 0
-10 < 0 && 20 > 0
T T

// b=19, a=-10

-10 + 19
9

// output: 9

Q: Write a program in C to print grades as per following criteria for obtained percentage of marks M out of 100 (AKTU PYQ)

(AKTU)

Obtained Percent Marks (M)	Grade
$90 < M \leq 100$	A^+
$80 < M \leq 90$	A
$70 < M \leq 80$	B^+
$60 < M \leq 70$	B
$50 < M \leq 60$	C
$M \leq 50$	F

Condition 1
— 2
— 3
— 4
— 5

else

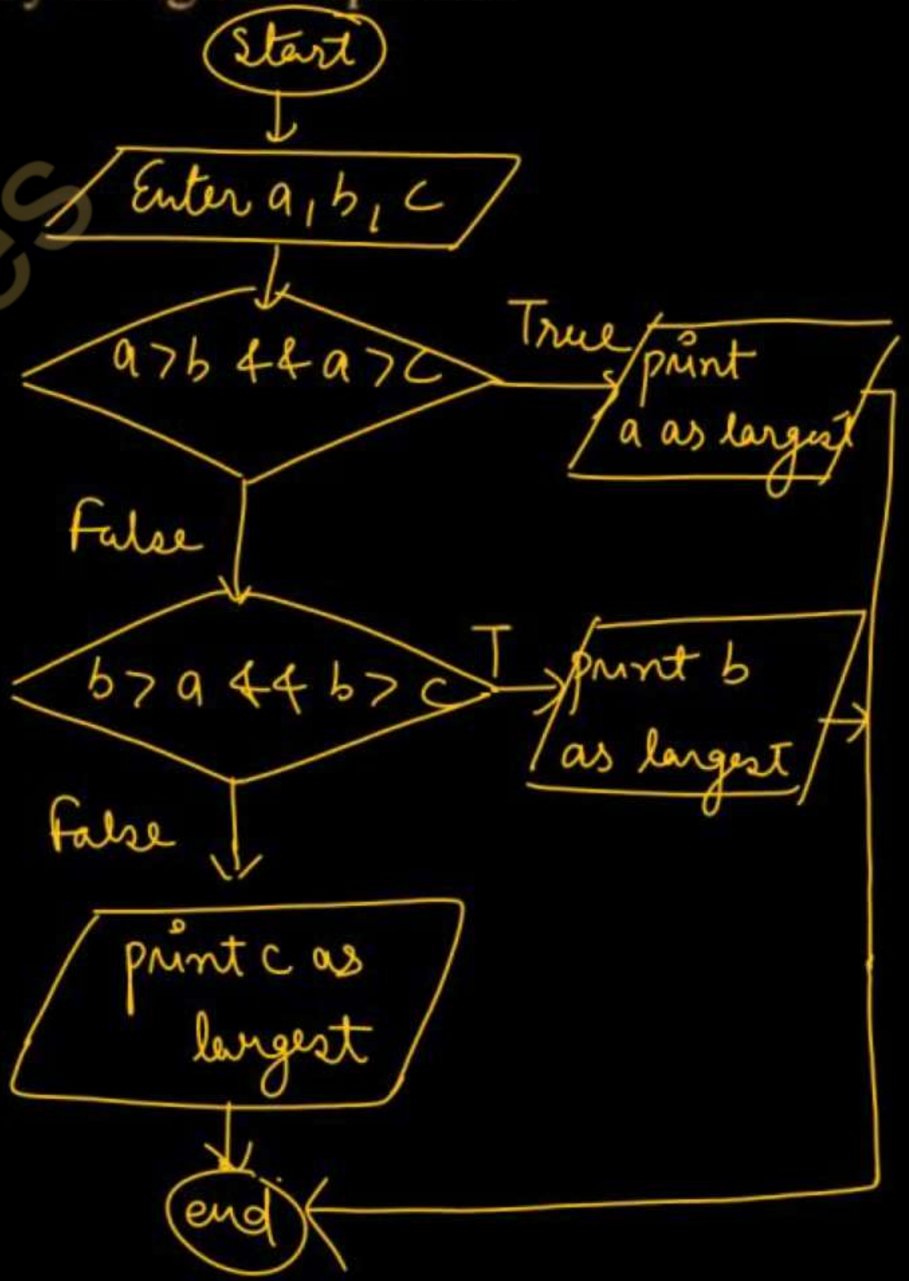
$90 < M \leq 100$

$(M > 90) \&\& (M <= 100)$

```
#include<stdio.h>
int main()
{
  float M;
  printf("Enter the percentage of marks");
  scanf("%f",&M);
  if((M>90)&&(M<=100))
  {
    printf("Grade A+");
  }
  else if((M>80)&&(M<=90))
  {
    printf("Grade A");
  }
  else if((M>70)&&(M<=80))
    printf("Grade B+");
  else if((M>60)&&(M<=70))
    printf("Grade B");
  else if((M>50)&&(M<=60))
    printf("Grade C");
  else
    printf("Grade F");
  return 0;
}
```

Write a program to ~~Program to~~ find largest of the three numbers by using && operator.

```
#include<stdio.h>
int main()
{ int a,b,c;
 printf("Enter three Numbers");
 scanf("%d%d%d",&a,&b,&c);
 if(a>b && a>c)
    printf("%d is the largest",a);
 else if( b>a && b>c)
    printf("%d is the largest",b);
 else
    printf("%d is the largest",c);
 return 0;
}
```



AKTU PYQ

Q-1 Find the output

```
#include <stdio.h>
int main()
{
int a = -10, b = 20;
if(a > 0 && b < 0)
a++;
else if(a < 0 && b < 0)
a--;
else if(a < 0 && b > 0)
b--;
else
b--;
printf("%d\n",a + b);
return 0;
}
```

Q-2 Write a program in C to print grades as per following criteria for obtained percentage of marks M out of 100

Obtained Percent Marks (M)	Grade
$90 < M \leq 100$	A^+
$80 < M \leq 90$	A
$70 < M \leq 80$	B^+
$60 < M \leq 70$	B
$50 < M \leq 60$	C
$M \leq 50$	F

Q-3 Explain different types of control statements used in C

Q-4 A certain grade of steel is graded according to the following conditions:

i. Hardness must be greater than 50

ii. Carbon content must be less than 0.7.

iii. Tensile strength must be less than 5600

H > 50 ✓

CC < 0.7 ✓

TS < 5600 ✓

The grades are as follows:

Grade is 10 if all the three conditions are met. if (H > 50 && CC < 0.7 && TS < 5600)

Grade is 9 if condition (i) and (ii) are met else if (H > 50 && CC < 0.7)

Grade is 8 if condition (ii) and (iii) are met else if (CC < 0.7 && TS < 5600)

Grade is 7 if condition (i) and (iii) are met else if (H > 50 && TS < 5600)

Grade is 6 if only one condition is met. (OR operator)

Grade is 5 if none of the conditions are met.

Write a program, which will require the user to give values of hardness, carbon content and tensile strength of the steel under consideration and output the grade of the

AKTU

B.Tech I-Year

GW GATEWAY CLASSES

PPS: Programming For Problem Solving

UNIT-2

Lecture- 7

Topics to be Covered

- WAP to check a character is uppercase or lowercase
- WAP to check entered character is alphabet or digit or special symbol
- WAP to calculate gross salary from basic salary according to given conditions

By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

Video Lectures I Pdf Notes I PYQs

C program to check whether a character is uppercase or lowercase.

```
#include<stdio.h>
int main()
{
  char ch;
  printf("Enter the character");
  scanf("%c",&ch);
if((ch>='a')&&(ch<='z'))
  { printf("%c is in lowercase",ch);
  }
else if((ch>='A')&&(ch<='Z'))
  {printf("%c is in uppercase",ch);
  }
  else printf("%c is not a letter of English alphabet",ch);
  return 0;
}
```

A – Z

a – z

Write a program to check whether the entered character is an alphabet (letter) or digit or special symbol.

```
#include<stdio.h>
int main()
{ char ch;
printf("Enter the character");
scanf("%c",&ch);
if(((ch>='a')&&(ch<='z'))||((ch>='A')&&(ch<='Z')))
    printf("%c is an alphabet",ch);
else if((ch>='0')&&(ch<='9')) // ch is between 0-9
    printf("%c is a digit",ch);
else
    printf("%c is a special symbol",ch);
return 0;
}
```

C program to input basic salary of an employee and calculate gross salary according to given conditions.

else

else if

if

Basic Salary <= 10000 : HRA = 20%, DA = 80%

Basic Salary is between 10001 to 20000 : HRA = 25%, DA = 90%

Basic Salary >= 20001 : HRA = 30%, DA = 95%

Basic Salary ⇒ bs

Gross Salary ⇒ gs

HRA ⇒ hra

DA → da

gs = bs + hra + da

```c
#include<stdio.h>
int main()
{ float bs, hra, da, gs;
  printf("Enter basic salary");
  scanf("%f",&bs);
if(bs>=20001)
  {
    hra= bs*0.30;
    da= bs*0.95;
  }
else if(bs>=10001 && bs<=20000)
  {
    hra= bs*0.25;
    da= bs*0.90;
  }
else
  {
    hra= bs*0.20;
    da= bs*0.80;
  }
  gs= bs+da+hra;
printf("The gross salary is= %0.2f",gs);
  return 0;
}
```

35.30956

it will show the value till two decimal places.

AKTU

B.Tech I-Year

GW GATEWAY CLASSES

PPS: Programming For Problem Solving

UNIT-2

Lecture- 8

Topics to be Covered

1. WAP to check if the given number is buzz number or not. (AKTU 2023-24)
2. C program to calculate energy bill. Read the starting and ending meter readings. The charges are based on given conditions. (AKTU 2023-24)
3. WAP which require the user to give values of hardness, carbon content and tensile strength of the steel under consideration and output the grade of the steel. (AKTU 2018-19)
4. WAP to calculate roots of quadratic equation
5. WAP to check triangle is valid or not if sides are given

Video Lectures I Pdf Notes I PYQs

By Dr. Nidhi Parashar Ma'am
- M.Tech Gold Medalist
- Net Qualified

Write a program to check whether the given number is buzz number or not. A number is said to be Buzz Number if it ends with 7 OR is divisible by 7. (AKTU 2023-24)

buzz Number

3167 ✓

63 ✓

77 ✓

n ends with 7 : `(n%10==7)`

n is divisible by 7 : `(n%7==0)`

`(n%10==7) || (n%7==0)`

638

10) 638 (4

60

88

80

8

7

digit present at one's place.

10) 3167 (316

30

16

10

67

60

7

3167 %10

316 (7)

ten's place

one's place

```c
#include stdio.h>
int main()
{ int n;
printf("Enter the number");
scanf("%d",&n);
if((n%10==7)||(n%7==0))
{ printf("%d is a buzz number",n);
}
else
{ printf("%d is not a buzz number",n);
}
return 0;
}
```

n = 837 ✓

(837%10==7) || ____

(T)

n = 78 ✗

(78%10==7) || (78%7==0)

(F) (F)

F

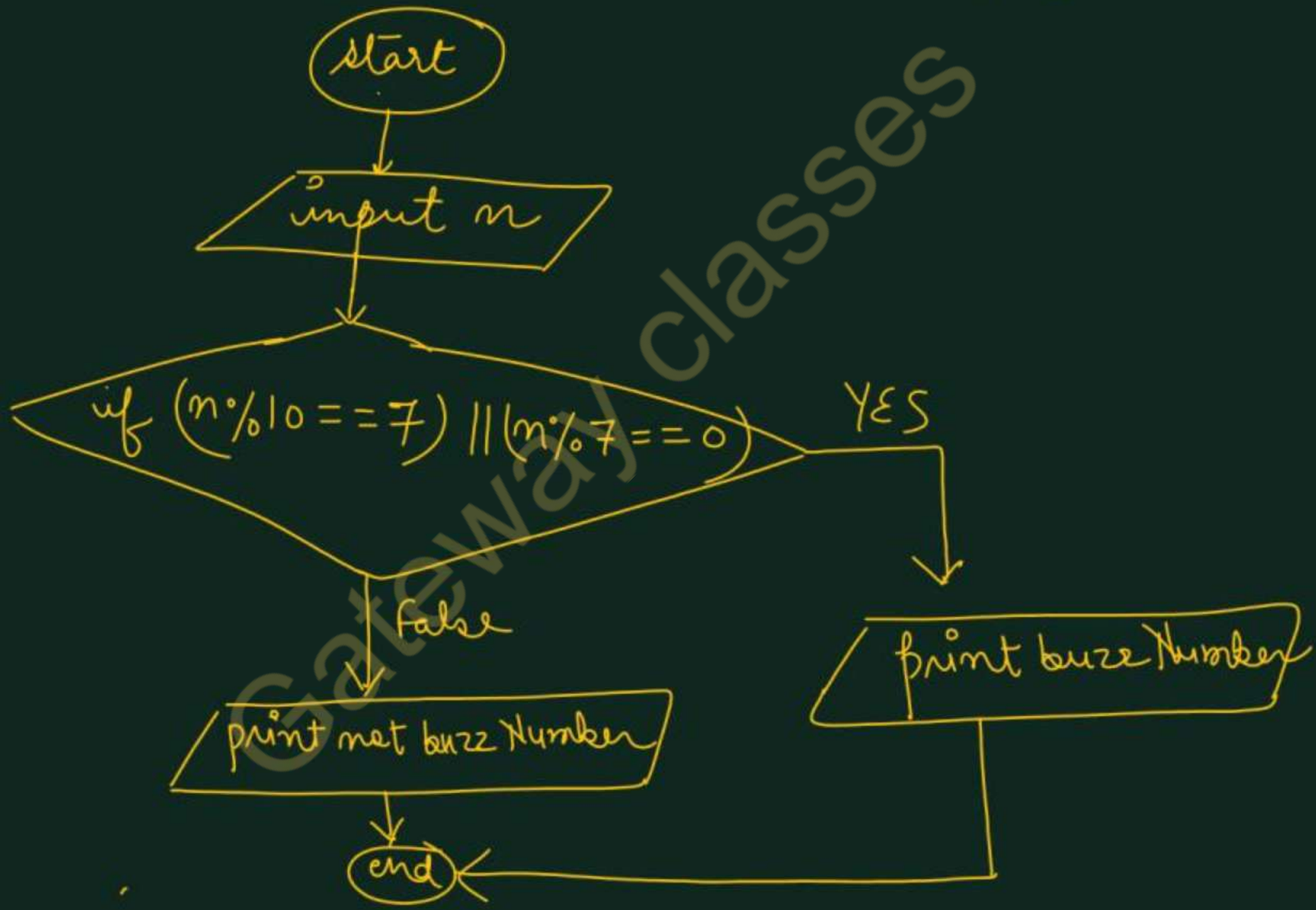
n = 63 ✓

(63%10==7) || (63%7==0)

(F) T

T

Flowchart of buzz Number program



C program to calculate energy bill. Read the starting and ending meter readings.The charges are as follows. (AKTU 2023-24)

No. of Units	Consumed rates in Rs.
0<units<=200	Rs 5.50 per Unit
200<units<=400	Rs 700 + Rs 6.0 per unit excess of 200
400<units<= 600	Rs 1400 + Rs 7.50 per unit excess of 400
600<units	Rs 1850+ Rs 9.0 per Unit excess of 600

bill = 1850 + (units − 600) * 9.0

```
#include<stdio.h>
int main
{ int units; float bill; sr, er;
 printf("Enter the no. of units");
 scanf("%d", &units); scanf("%d %d", &sr, &er);
                      units = er - sr;
 if( units > 600)
 {
     bill = 1850 + (units-600)*9.0;
 }
else if( units >400 && units <=600)
  {
     bill = 1400 + (units-400)*7.50;
  }
```

units = 500

```
else if ( units >200 && units <= 400)
 {
      bill = 700 +(units-200)*6.0 ;
 }
else if (units >0  && units <=200)
  {
       bill =  units *5.50;
  }
else
      {  printf("Invalid units for bill generation");
      }
printf("The energy bill is=%f", bill);
 return 0;
}
```

A certain grade of steel is graded according to the following conditions:

i.Hardness must be greater than 50 $h > 50$ (1)

ii.Carbon content must be less than 0.7. $cc < 0.7$ (2)

iii.Tensile strength must be less than 5600 $ts < 5600$ (3)

The grades are as follows:

Grade is 10 if all the three conditions are met. $(h > 50 \&\& cc < 0.7 \&\& ts < 5600)$

Grade is 9 if condition (i) and (ii) are met $(h > 50 \&\& cc < 0.7)$

Grade is 8 if condition (ii) and (iii) are met $(cc < 0.7) \&\& (ts < 5600)$

Grade is 7 if condition (i) and (iii) are met $(h > 50) \&\& (ts < 5600)$

Grade is 6 if only one condition is met. $(h > 50) \,||\, (cc < 0.7) \,||\, (ts < 5600)$

Grade is 5 if none of the conditions are met. $(h <= 50) \&\& (cc >= 0.7) \&\& (ts >= 5600)$

Write a program, which will require the user to give values of hardness, carbon content and tensile strength of the steel under consideration and output the grade of the steel.(AKTU 2018-19)

```
#include <stdio.h>
int main()
{
 int h, ts;
 float cc;
printf("Enter hardness, tensile strength
          and carbon content ");
scanf("%d%d%f", &h, &ts, &cc);
if((h>50)&&(cc<0.7)&&(ts<5600))
{ printf("Grade 10\n");
}
else if((h>50)&&(cc<0.7))
{
  printf("Grade 9\0");
}
else if((cc<0.7)&&(ts<5600))
{ printf("Grade 8\n");
}
else if((h>50)&&(ts<5600))
{ printf("Grade 7\n");
}
else if((h>50)||(cc<0.7)||(ts<5600))
{ printf("Grade 6\n");
}
else if((h<=50)&&(cc>=0.7)||(ts>=5600))
{ printf("Grade 5\n");
}
else { printf("Invalid option");
}
} return 0;
```

Write a program to calculate roots of quadratic equation.

$$ax^2 + bx + c = 0$$

determinant $(d) = b^2 - 4ac$

$d = b*b - 4*a*c$

1. $d > 0$ → Real and distinct Roots

$$\text{Root 1} = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$$

$$\text{Root 2} = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

2. $d = 0$ → Real and equal roots

$$\text{Root 1} = -b/2a$$

$$\text{Root 2} = -b/2a$$

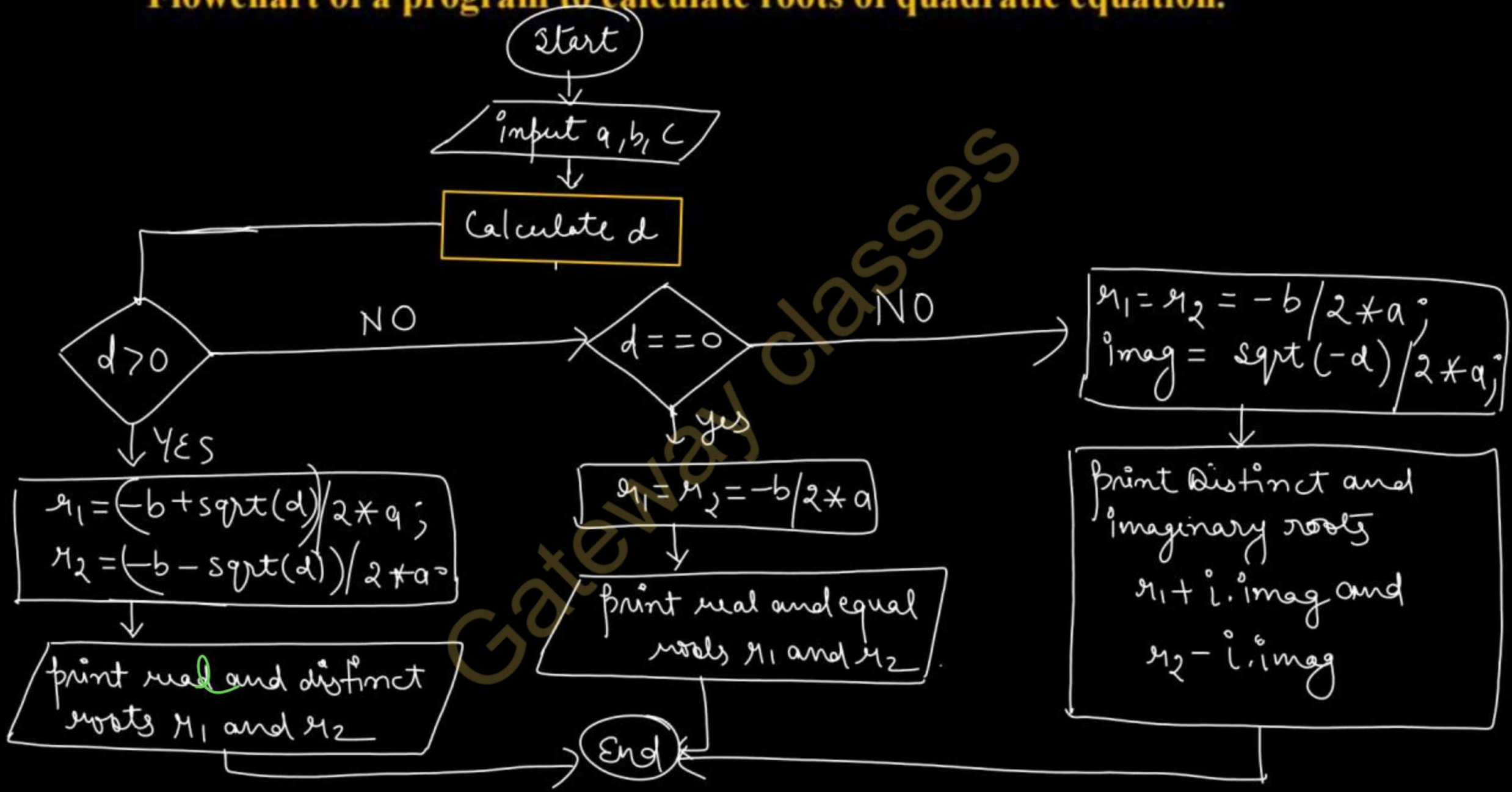
3. $d < 0$ → Imaginary and distinct roots

$$\text{Root 1} = \frac{-b}{2a} + i\frac{\sqrt{-(d)}}{2a}$$

$$\text{Root 2} = \frac{-b}{2a} - i\frac{\sqrt{-(d)}}{2a}$$

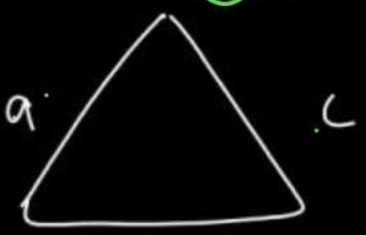
```
#include<stdio.h>
#include<math.h>
int main()
{
    int a, b, c, d;
    float r1, r2, imag;
    printf("Enter a, b, c values");
    scanf("%d%d%d", &a, &b, &c);
    d = (b*b) - (4*a*c);
    if (d > 0)
    {
        r1 = (-b + sqrt(d)) / 2*a;
        r2 = (-b - sqrt(d)) / 2*a;
        printf("Real and distinct roots:
               %f and %f", r1, r2);
    }
    else if (d == 0)
    {
        r1 = r2 = -b / 2*a;
        printf("Real and equal roots: %f and %f",
                                    r1, r2);
    }
    else
    {
        r1 = r2 = -b / 2*a;
        imag = sqrt(-d) / 2*a;
        printf("Distinct and Imaginary Roots:
               %f + i.%f and %f - i.%f", r1, imag,
                                    r2, imag);
        return 0;
    }
```

Flowchart of a program to calculate roots of quadratic equation.

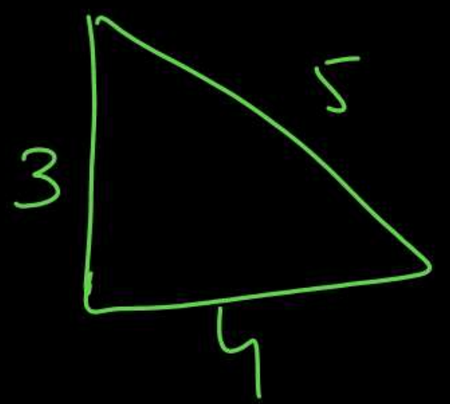


C program to check whether triangle is valid or not if sides are given. (Imp)

```
#include<stdio.h>
int main()
{ int a,b,c;
printf("Enter sides");
scanf("%d %d %d",&a,&b,&c);
if(((a+b)>c)&&((b+c)>a)&&((a+c)>b))
{
printf("The triangle is valid");
}
else{
printf("The triangle is not valid");
}
return 0;
}
```



a	b	c
3	4	5



AKTU PYQ

1. WAP to check if the given number is buzz number or not. (AKTU 2023-24)
2. C program to calculate energy bill. Read the starting and ending meter readings. The charges are based on given conditions. (AKTU 2023-24)
3. WAP which require the user to give values of hardness, carbon content and tensile strength of the steel under consideration and output the grade of the steel. (AKTU 2018-19)

AKTU

B.Tech I-Year

GATEWAY CLASSES

PPS: Programming For Problem Solving

UNIT-2

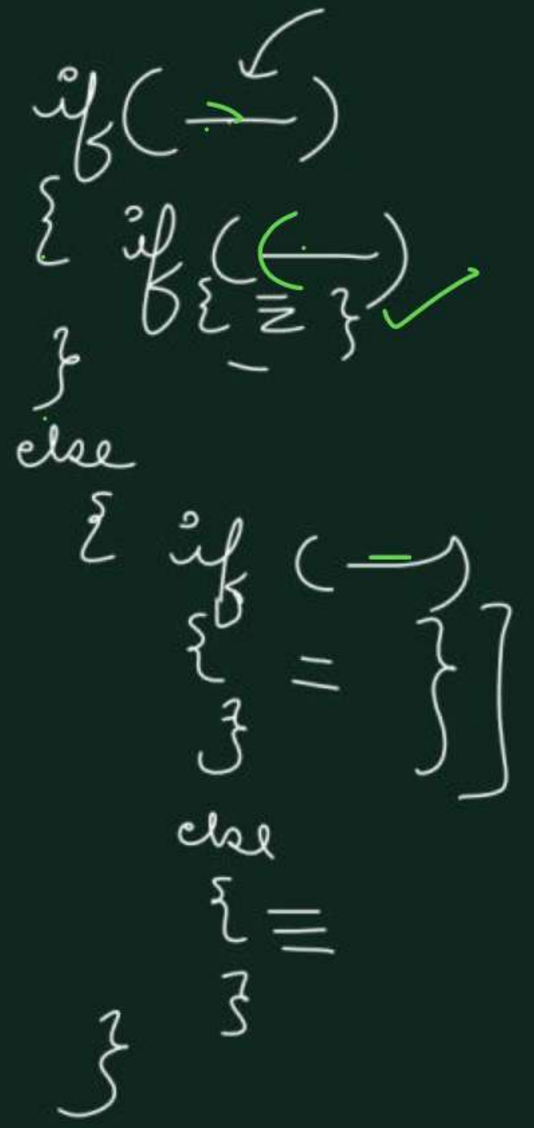
Lecture- 9

Topics to be Covered

- Nested-if-else statement
- WAP to check the triangle with given sides is equilateral or isosceles or scalene
- WAP to check the triangle with given angles is right angled or equilateral or isosceles or scalene
- WAP to print the largest number among three numbers using nested if-else.
- AKTU PYQs

Video Lectures I Pdf Notes I PYQs

By Dr. Nidhi Parashar Ma'am
- M.Tech Gold Medalist
- Net Qualified



Nested if-else Statement

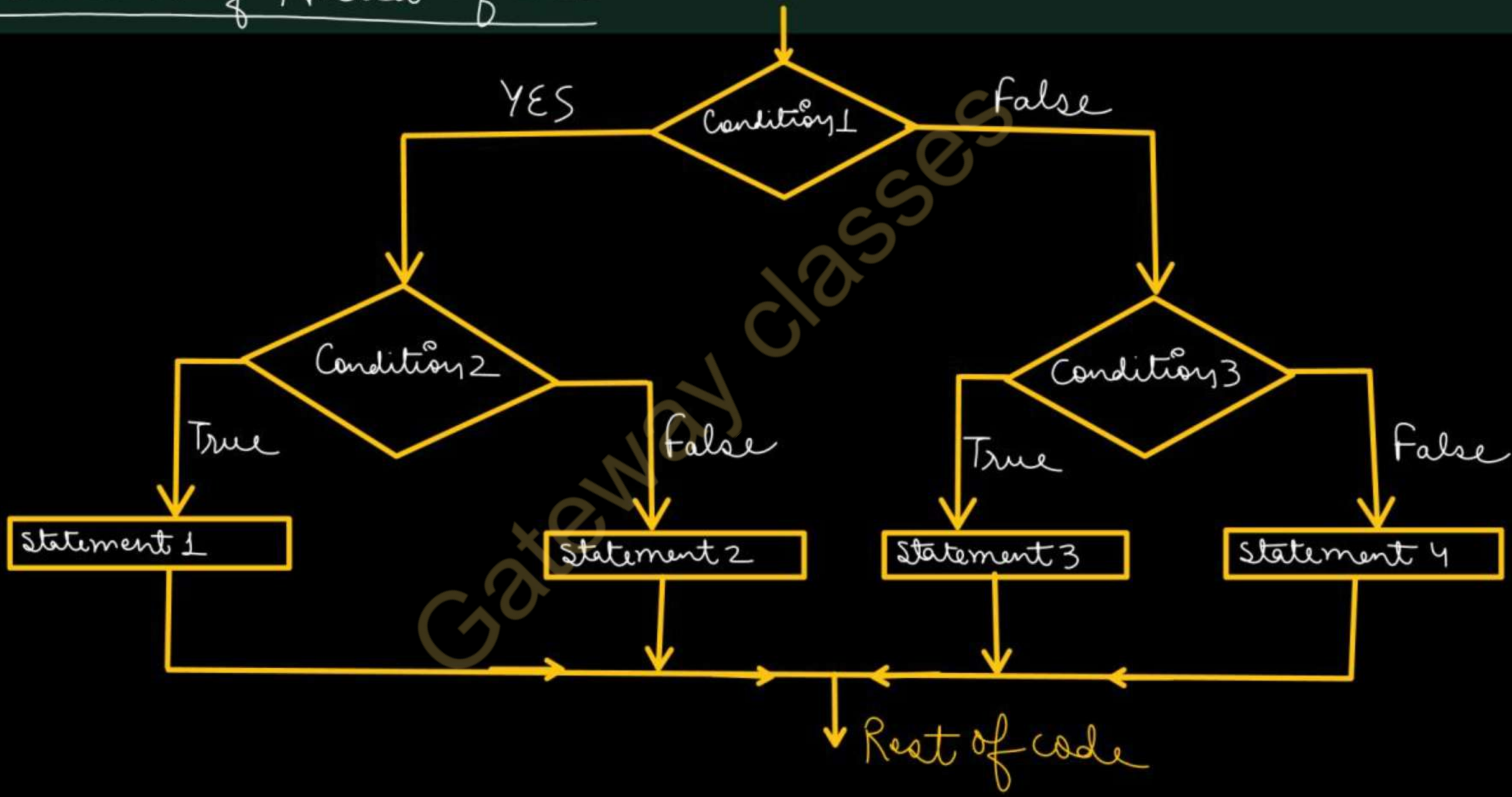
- An entire *if-else* construct can be written within either the body of the *if* statement or the body of an *else* statement. This is called nested if-else.

- It first tests Condition1, if this condition is TRUE then it tests Condition2, if Condition2 evaluates to TRUE then Statement1 is executed otherwise Statement2 will be executed.

- if Condition1 is FALSE then it tests Condition3, if Condition3 evaluates to TRUE then Statement3 is executed otherwise Statement4 is executed.

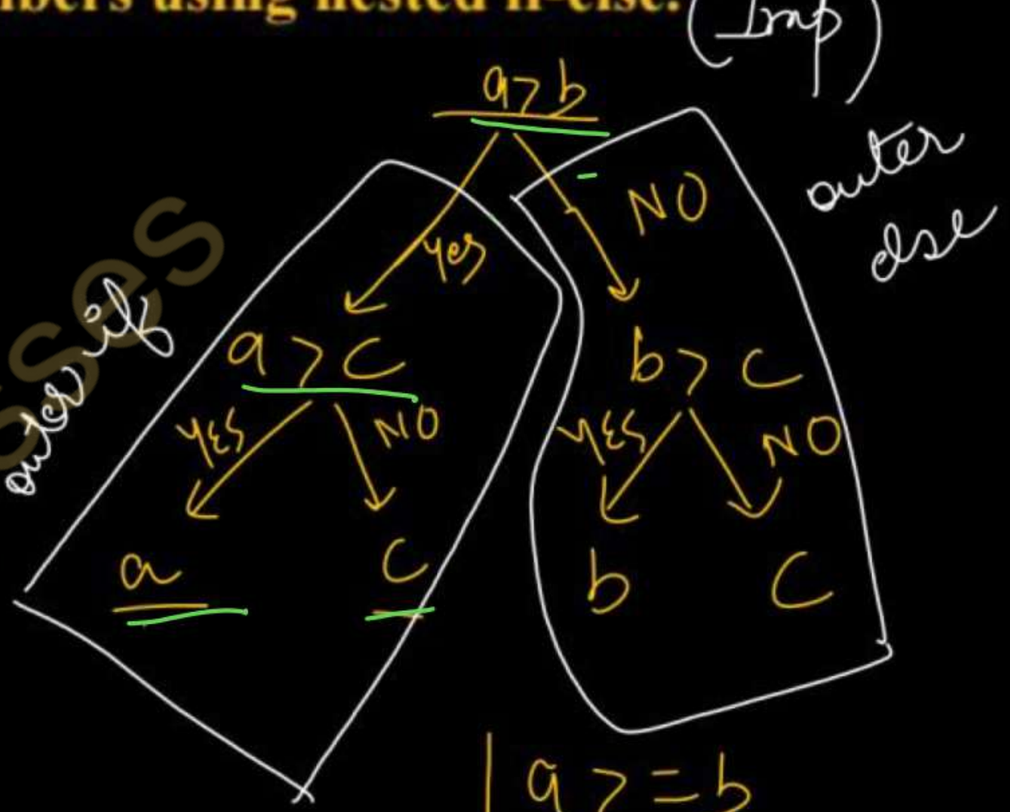
```
if (Condition1)
{
        if(Condition2)
        {
                Statement1;
        }
        else
        {
                Statement2;
        }
}
else
{
        if(Condition3)
        {
                Statement3;
        }
        else
        {
                Statement4;
        }
}
```

Flow chart of Nested if-else



➢ WAP to print the largest number among three numbers using nested if-else. (Imp)

```
#include<stdio.h>
int main()
{ int a, b, c;
printf("Enter three numbers");
scanf("%d%d%d", &a, &b, &c);
if(a>b)
{
  if(a>c)
  { printf("%d is the largest", a);
  }
  else
  { printf("%d is the largest", c);
  }
}
```



a>=b
a>=c

```
else
{   if( b>c)
    {
      printf ( "%d is the largest", b);
    }
    else
    {
      printf ( "%d is the largest", c);
    }
}

    return 0;
}
```

Write a program to ~~Program to~~ check whether the triangle with given sides is equilateral or isosceles or scalene.

```
#include<stdio.h>
int main()
{ int a, b, c;
  printf("Enter three sides of triangle");
  scanf("%d%d%d", &a, &b, &c);
  if(((a+b)>c)&&((b+c)>a)&&((a+c)>b))
  {
    if((a==b)&&(b==c))
    {
      printf("Equilateral Triangle");
    }
```

a b c

Equilateral → a, b, c are equal

Isoceles → any two sides are equal

Scalene → All sides are distinct

```
else if( (a==b) || (b==c) || (a==c) )
{
    printf("Isoceles Triangle");
}
else
{
    printf("Scalene Triangle");
}
} // Ending of outer if
else
{
    printf("Invalid Triangle");
}
return 0;
}
```

a, b, c sides are given

(1) $c^2 = a^2 + b^2$

OR

(2) $a^2 = c^2 + b^2$

OR

(3) $b^2 = a^2 + c^2$

WAP to check if a given triangle is right angled or not if sides are given

```
#include<stdio.h>
int main()
{ int a,b,c;
 printf("Enter sides of a triangle");
 scanf("%d%d%d",&a,&b,&c);
 if((a+b)>c && (b+c)>a && (a+c)>b)
 {
    if(((c*c=a*a+b*b)||(a*a=b*b+c*c)||(b*b=a*a+c*c))
    { printf("The triangle is right angled");
    }
 }
else{ printf("Triangle is Invalid");}
 return 0;
}
```

Write a program to check whether the triangle with given angles is right angled or equilateral or isosceles or scalene.

$a+b+c=180$

$a/b/c=90$

$a=b=c=60$

```
#include<stdio.h>
int main()
{ int A,B,C;
printf("Enter angles of a triangle");
scanf("%d%d%d",&A,&B,&C);
if((A+B+C)==180) // validity of triangle condition
{
  if((A==90)||(B==90)||(C==90))
    { printf(" Right Angled Triangle");
    }
```

```
else if( (A==60) && (B==60) && (C==60) )
{
  printf( "Equilateral Triangle ");
}
else if( (A==B) || (B==C) || (A==C) )
{
  printf( " Isoceles Triangle");
}
else
{
  printf( " Scalene Triangle");
}
} // Ending of Inner if-else ladder
```

```
else
    {
        printf("The triangle is Invalid");
    }
    return 0;
}
```

Ex. A = 80, B = 60, C = 40

AKTU

B.Tech I-Year

GW GATEWAY CLASSES

PPS: Programming For Problem Solving

UNIT-2

Lecture- 10

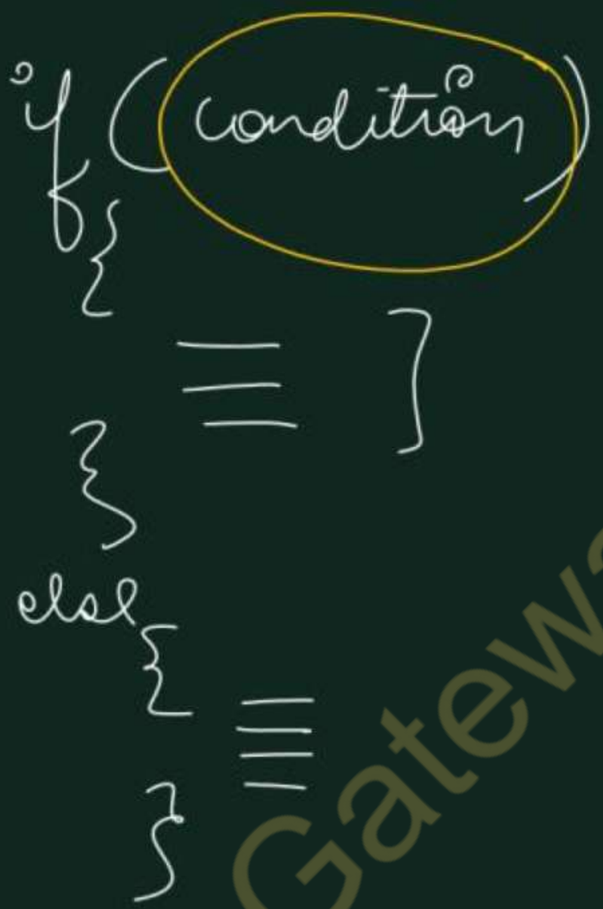
Topics to be Covered

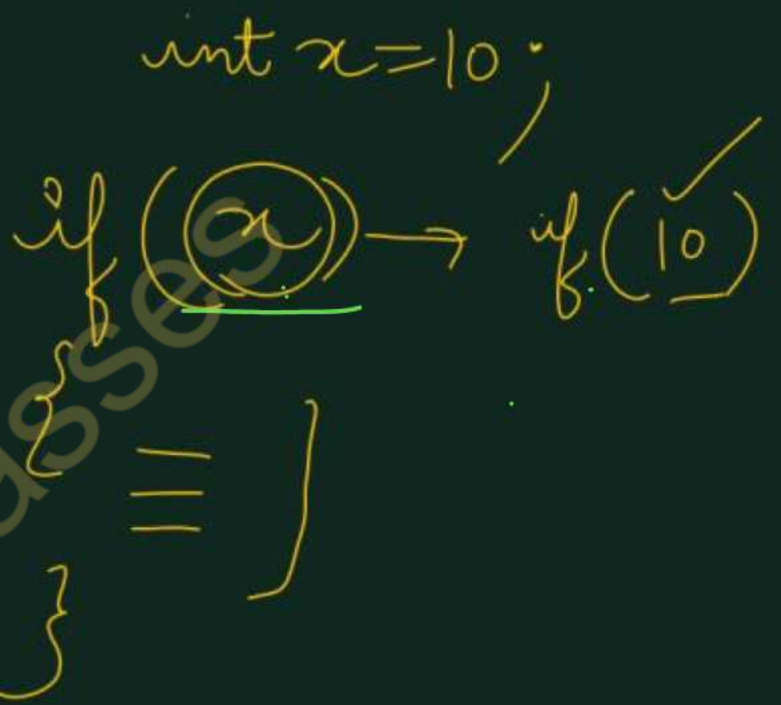
- Switch case with programs

By Dr. Nidhi Parashar Ma'am

- M.Tech Gold Medalist
- Net Qualified

Video Lectures I Pdf Notes I PYQs





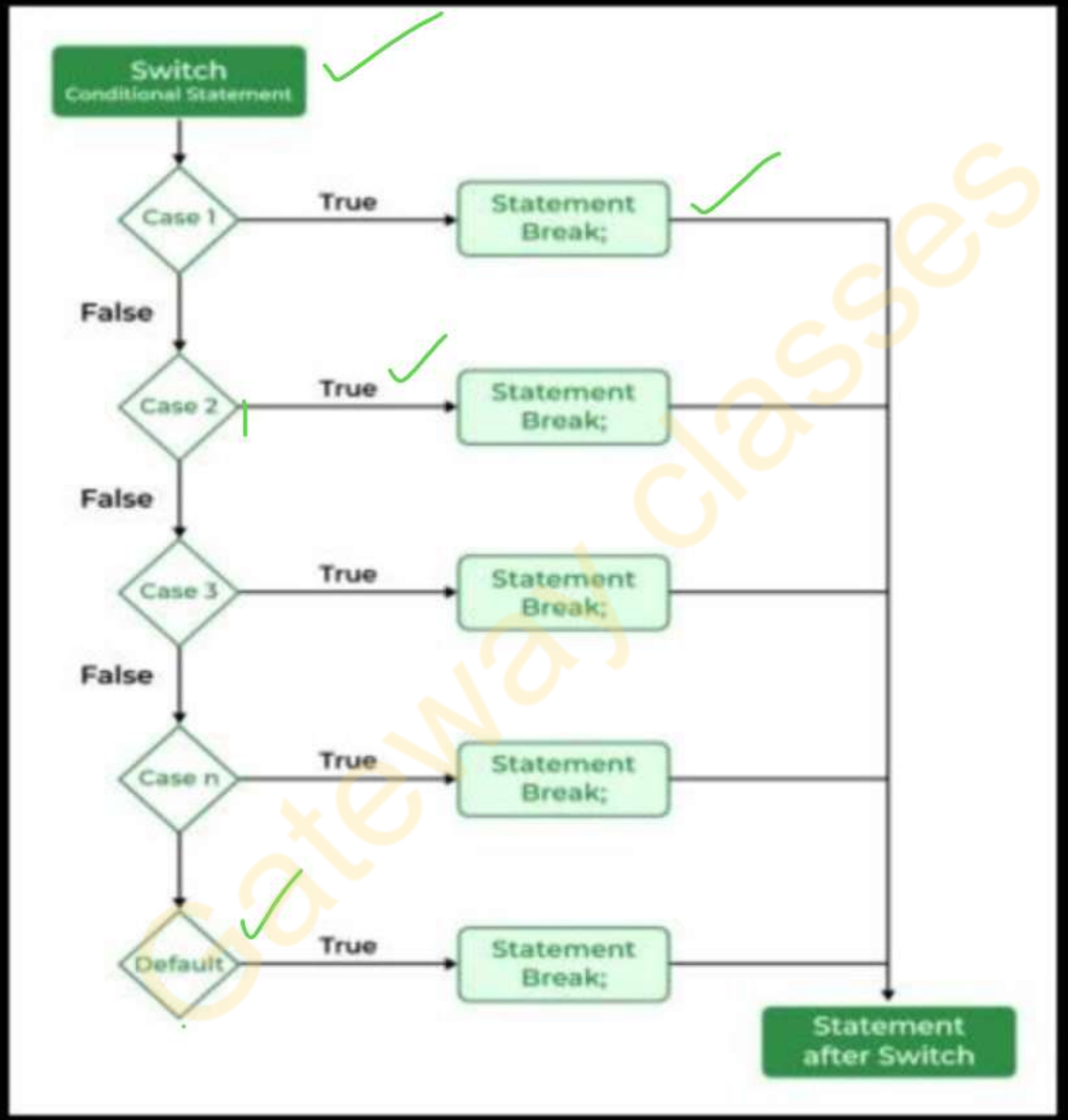
Switch case

- It evaluates a given expression and based on the evaluated value, it executes the statements associated with it.
- Once the case match is found, a block of statements associated with that particular case is executed. Each case in a block of a switch has a different name/number which is referred to as an identifier.
- The value provided by the user is compared with all the cases inside the switch block until the match is found. If a case match is not found, then the default statement is executed, and the control goes out of the switch block.

Syntax

```
switch(expression)
{
case value1: statement_1;
             break;
case value2: statement_2;
             break;
..
case value_n: statement_n;
             break;
default:     default_statement;
}
```

Flowchart of switch case Statement



break
default

Rules for using the switch:

- Case Label must be unique and end with Colon.
- Case labels must have constants / constant expression.
- Case label must be of Integer/ Character and should not be float.
- Switch case should have at most one default label.
- Default label is Optional and can be placed anywhere in the switch.
- Break Statement takes control out of the switch.
- **Two or more cases may share one break statement (The break statement is optional. If omitted, execution will continue on into the next case. The flow of control will fall through to subsequent cases until a break is reached.)**
- Nesting (switch within switch) is allowed.
- Relational Operators are not allowed in Switch Statement.
- Empty Switch case is allowed.

```
int x = 10;
switch (x)
{
    case 15 : printf ("Hi");
              break;
    case 10 : printf ("Hello");
              break;
    case 20 : printf ("world");
              break;
    default : printf ("Bye");
}
```

case matched

// This statement is executed

```
int x = 10;
switch (x)
{
case 1: printf("A");
        break;
case 2: printf("B");
        break;
default: printf("C");
}
```

output: C

should be int expression.

An expression that produces integer value after its evaluation.

should be integer / char constant

```
int x=50;
switch (x)
{
  case 10 : printf("A");
            break;
  case 'a' : printf("B");
             break;
  case 5 : printf("C");
           break;
}
```

Mixed types of case values (int / char) are allowed.

```
int x = 1;
switch (x)
{
    case 1: printf("Hi");
            break;

    case 2==3: printf("Bye");
               break;

}
```

⇒ case matched

```
int x = 1;
switch (x)
{
    case 1: printf ("Hello");
            break;
    case 3==3: printf ("world");
               break;
    default: printf("Bye");
}
```

duplicate cases ✗

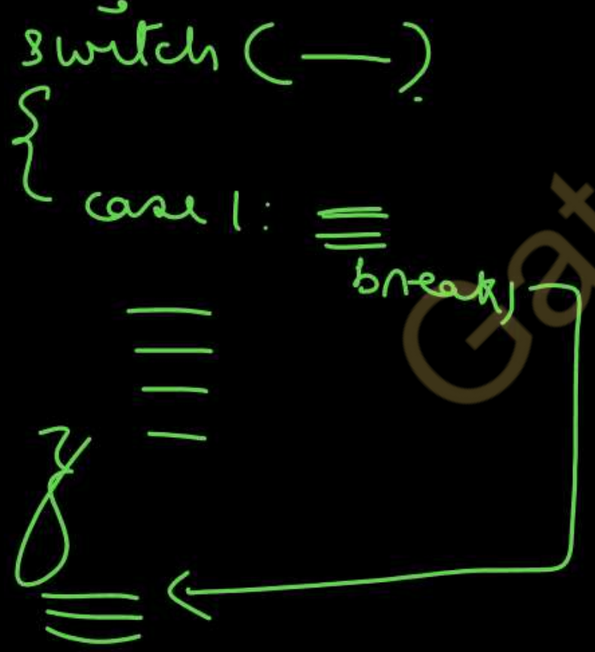
3==3 => 1

compilation Error as duplicate cases are not allowed.

~~Break~~ break in switch case (Important)

break is a Keyword.

- Used to stop the execution inside a switch block.
- When a break statement is reached, the switch terminates, and the flow of control jumps to the next line following the switch statement. (execution of)
- The **break statement is optional**.
- If omitted, execution will continue on into the next case. The flow of control will fall through to subsequent cases until a break is reached.



Switch case without break

```
int main ()
{
    int x=5;          8 ✓
    switch (x = x+3)
    {
        case 1: printf("A");
                break;
        case 5: printf("B");
                break;
        case 8: printf("C");
        case 10: printf("D");
        case 11: printf("E");
                break;
        default: printf("F");
}
```

Matched case → case 8

$x = x + 3$

$x = 5 + 3$

$x = 8$

All following cases are executed untill break is found.

This property is called as falling through property of switch.

Default in switch case (Important)

- The default keyword is used to specify the set of statements to execute if there is no case match.
- It is optional to use the default keyword in a switch case.
- Even if the switch case statement does not have a default statement, it would run without any problem.

```
int x=10;
switch(x)
{
default: printf("Hi");
break; X
case 1: printf("Bye");
}
```

output: Hi

Write a program to print days of the week.

```
#include <stdio.h>
int main()
{
    int day;
    printf("Enter day no. from 1-7");
    scanf("%d", &day);
    switch(day)
    {
        case 1: printf("Monday");
                break;
        case 2: printf("Tuesday");
                break;
        case 3: printf("Wednesday");
                break;
        case 4: printf("Thursday");
                break;
        case 5: printf("Friday");
                break;
        case 6: printf("Saturday");
                break;
        case 7: printf("Sunday");
                break;
        default: printf("Invalid day no.");
    }
}
```

v. Imp

Write a program to simulate simple calculator (Menu driven)

```
#include<stdio.h>
int main()
{   int a, b;
    char choice;
printf("Enter + for addition\n");
printf("Enter - for subtraction\n");
printf("Enter * for multiplication\n");
printf("Enter / for division\n");
printf("Enter % for remainder\n");
scanf("%c", &choice);
printf("Enter two numbers");
scanf("%d%d", &a, &b);
switch(choice)
{case '+': printf("The sum is = %d", (a+b));
          break;
case '-': printf("The subtraction is = %d", (a-b));
          break;
case '*': printf("The product is = %d", (a*b));
          break;
case '/': printf("The quotient is = %d", (a/b));
          break;
case '%': printf("The remainder is = %d", (a%b));
          break;
default : printf("Invalid choice");
}
}
```

```c
#include <stdio.h>
int main()
{
        int num = 2;
        switch (num + 2)
        {
    x   case 1:
                printf("Case 1: ");
    x   case 2:
                printf("Case 2: ");
    x   case 3:
                printf("Case 3: ");
        default:
                printf("Default: ");
        }
        return 0;
}
```

num + 2 → 4

output : Default

```
#include<stdio.h>
void main()
{
        int x = 1;
        switch (x << (2 + x))
        {
        default:
                printf(" A");
        case 4:
                printf(" B");
        case 5:
                printf(" C");
        case 8:
                printf(" D");
        }
}
```

$x << (2+x)$

↓

bitwise left shift

$= x << (2+1)$

$= x << 3$

$= x * 2^3$

$= 1 \times 8 \Rightarrow 8$

output : D

```
int main()
{
int a=3;
switch(a)
{
}
printf("MySwitch");
}
```

⇒ Empty switch is allowed

output: MySwitch

```
int main()
{
int a;
switch(a=1)
{
    printf("DEER ");
}
printf("LION"); }
```

⇒ switch (a = 1)
{
};
{
printf("DEER");
}
printf ("LION");

Empty switch case

This is not part of switch

output: DEER LION

```
if(0)
{ printf("Hi");
}
printf("Bye");
```

output : Bye

```
if(0);
{ printf("Hi");
}
printf("Bye");
```

output: Hi Bye

```
if(0)
;
{ printf("Hi");
}
printf("Bye");
```

not part of if

```
if(0);
{ printf("Hi");
}
else
{ printf("Bye");
}
```

```
if(0)
;
{ printf("Hi");
}
else
```

compile time error

```
int main()
{
int a;
switch(a=1);
{
    printf("DEER ");
    case 1:printf("hi ");
}
printf("LION"); }
```

switch statement is terminated

not part of switch

Compilation Error

```
int main()
{
int a=1;
switch(a=1)
{
    printf("DEER ");
    case 1:printf("hi ");
}
printf("LION"); }
```

will never be executed

output: hi LION

```
int main()
{
char code='K';
switch(code)
{
        case 'A': printf("ANT ");
                  break;
        case 'K': printf("KING ");
                  break;
        default: printf("NOKING");
}
printf("PALACE");
}
```

output : KING PALACE

```
int main()
{
char code='A';
switch(code)
{
        case 64+1: printf("ANT ");
                   break;
        case 8*8+4: printf("KING ");
                   break;
        default: printf("NOKING");
}
printf("PALACE"); }
```

Ascii (A) → 65

'A'

65

68

ASCII value of 'A' is = 65

output: ANT PALACE

```
int main()
{
int k=64;
switch(k)
{
case k<64: printf("SHIP ");
           break;
case k>=64: printf("BOAT ");
           break;
default: printf("PETROL");
}
printf("CHILLY"); }
```

case numbers are invalid.

Compilation Error

```
int main()
{
int k=8;
switch(k)
{
case 1==8: printf("ROSE ");
        break;
case 1 && 2: printf("JASMINE ");
        break;
default: printf("FLOWER ");
}
printf("GARDEN");
}
```

output: FLOWER GARDEN

```
int main()
{
int k=1;
switch(k)
{
case 1==8: printf("ROSE ");
        break;
case 1 && 2: printf("JASMINE ");
        break;
default: printf("FLOWER ");
}
printf("GARDEN");
}
```

output : JASMINE GARDEN

```
int main()
{
int k=1;
switch(k)
{
case 1==1: printf("ROSE ");
        break;
case 1 && 2: printf("JASMINE ");
        break;
default: printf("FLOWER ");
}
printf("GARDEN");
}
```

duplicate case numbers.

⇒ Compilation Error

```
int main()
{
int k=1;
switch(k)
{
case 1==1: printf("ROSE ");
           break;
case !0: printf("JASMINE ");
           break;
default: printf("FLOWER ");
}
printf("GARDEN");
}
```

duplicate cases

compilation error

```
int main()
{
int k=1;
switch(k)
{
case 1<11: printf("ROSE ");
        break;
case !8: printf("JASMINE ");
        break;
default: printf("FLOWER ");
}
printf("GARDEN");
}
```

!8

!(T)

⇒ false ⇒ 0

output: ROSE GARDEN

```
int main()
{ int k=0;
switch(k)
{
case 1<11: printf("ROSE ");
        break;
case !k: printf("JASMINE ");
        break;
default: printf("FLOWER ");
 }
printf("GARDEN");
}
```

k = 0

1

!k

!0

!(False)

True

⇒ 1

duplicate case

Compilation Error

```
int main()
{
switch(24.5)
{
case 24.5: printf("SILVER ");
        break;
case 25.0: printf("GOLD ");
        break;
default: printf("TIN ");
}
printf("COPPER");
}
```

Compilation Error

Because float values can't be used as case numbers and switch expression can't be a floating point expression.

Imp **Difference between if-else and switch-case:**

Switch case	If-else
Syntax: ✓ + flowchart	Syntax : ✓ flowchart
Fast in operation as it directly goes to the matching case for execution.	Slow in operation because the program control may not directly reach to the specific condition.
Takes only integer in expression or case to match.	It can compare for integer as well as float value.
Can test only for equality(= =).	Can evaluate all relational or logical expression.
An expression is evaluated and the code is selected on the value of the expression.	Code is selected based on the truth value of the expression.
Require break statement to avoid execution the block just below the current block.	It does not require break statement.

→ Use default

doesn't use default

Write a program to check whether the character is vowel or not using switch case.

```
#include<stdio.h>
int main()
{ char ch;
printf("Enter the character");
scanf("%c",&ch);
switch(ch)
{
 case 'a':
 case 'e':
 case 'i':
 case 'o':
 case 'u':
 case 'A':
 case 'E':
 case 'I':
 case 'O':
 case 'U': printf("%c is vowel",ch);
           break;
 default: printf("%c is consonant",ch);
}
}
```

Difference contdd.....

if-else	Switch case
→ Misplaced else problem.	→ doesn't have misplaced else error problem
→ Multiple conditions can be combined.	→ can't be combined.
→ So much complexity arises due to lot of parenthesis.	→ only one pair of parenthesis is required.
→ As many conditions may be checked as required.	→ May not be feasible to write all the cases.

AKTU PYQs

1. Write a program to discuss the use of break statement in switch statement.

(AKTU 2023-24) (2021-22) (2020-21)

2. How to use break statement in C? Explain with some sort of code. (AKTU 2018-19)

3. What is the use of break statement? Write a program to check whether character is vowel or not.

4. Write limitations of switch case. (AKTU 2021-22)

5. Compare switch case with if-else ladder. Write menu driven program to perform basic functions of calculator. (AKTU 2021-22)

6. What is case control structure in C.? What is the reason for using break statement at the end of each case in case control block? (AKTU 2018-19)

7. Explain the use of default in switch statement. Write a program that takes two operands and one operator from the user and perform the operation and prints the result by using switch statement.

(AKTU 2018-19)